Pavel A. Stepanov, pavel@stepanoff.info
Степанов П.А.

pavel@stepanoff.me

Технологии разработки серверных информационных систем

Методическое пособие

ГУАП

Санкт-Петербург

-2026-

Pavel A. Stepanov, pavel@stepanoff.info © 2018 -2023
Кафедра компьютерных технологий и программной инженерии (№43) СПб ГУАП

Оглавление

Введение 5

График выполнения лабораторных работ 6

Примерные варианты заданий 7

Технические требования 8

Требования к отчету 9

Конфигурирование рабочего места 10

- Требования к рабочему месту 10
- Установка Java 10
- Установка IDE 10
- Краткое описание системы сборки Maven 10

Лабораторная работа №1. Разработка простого серверного приложения с использованием сервлетов и фильтров 12

- Протокол http. 12
- Cookie. 15
- Http сессия. 16
- Архитектура простого веб-приложения. 17
- Структура класса сервлета. 19
- Структура класса фильтра. 20
- Задание на лабораторную работу. 21

Лабораторная работа №2. Разработка ресурса REST/JSON сервиса 22

- Предпосылки появления Web API 22
- SOAP 22
- JSON 23
- RESTful API 24
- Инструменты разработчика 26
- Реализация REST средствами Spring-boot 26
- Развитие REST – Query Data API 28
- Формальные описания интерфейсов 29
- Задание на лабораторную работу. 32

Лабораторная работа №3. Разработка простого AJAX приложения Spring 33

- Архитектура клиентского представления. 33
- Технология Ajax. 33
- Библиотека AngularJS. 34

Задание на лабораторную работу. 36
Лабораторная работа №4. Разработка формы логина 37
Основы авторизации и аутентификации. Виды аутентификации. 37
Виды атак на веб-приложение. 37
Снифферы и атаки “Man-in-the-middle”. 38
Защита от атак XSS и CSRF. 40
Хеширование паролей. 41
Basic и Digest аутентификация 42
Аутентификация через директории пользователей (Kerberos, LDAP). 43
Внешняя аутентификация через клиент (OpenID, OAuth). 43
SSO аутентификация 45
Реализация аутентификации и авторизации средствами Spring. 45
Задание на лабораторную работу. 50
Лабораторная работа №5. Разработка приложения с использованием thymeleaf 51
Шаблоны страниц JSP. 51
Spring MVC 52
Шаблоны страниц Thymeleaf 52
Задание на лабораторную работу. 55
Лабораторная работа №6. Разработка приложения с использованием Hibernate 56
Основные понятия JDBC. Соединение с СУБД. 56
JDBC Template. 58
ORM. Виды ORM. Стандарт JPA. 60
Поддержка ORM в Spring. 62
Поддержка транзакций 64
Паттерн DTO. 66
Задание на лабораторную работу. 67
Лабораторная работа №7. Разработка приложения с асинхронной очередью сообщений 68
Архитектура, управляемая событиями (EDA, Event Driven Architecture). 68
Брокеры сообщений. Основные модели обмена сообщениями. 69
RabbitMQ. 70
Kafka 71
Задание на лабораторную работу. 73
Лабораторная работа №8. Разработка микросервиса 75

Виртуализация и связь с автоматическим развертыванием. 75
Контейнеризация. Docker. 76
Dockerfile. 77
Docker desktop. 80
Задание на лабораторную работу. 82
Список литературы. 83
Интернет-источники. 83

Введение

Предмет “Технологии разработки серверных информационных систем” призван дать обучающимся представление о методах построения современных приложений, размещаемый в сети Интернет (интранет). Лабораторные работы по этому предмету знакомят обучающихся со следующими технологиями:

- Общее представление о современных тенденциях в разработке для web
- Структура веб-серверов, фильтры и сервлеты, сессии
- Интерфейсы JSON и SOAP. Подход REST.
- инъекция зависимостей и инверсия управления (DI, IoC и их реализация в Spring)
- аспектно-ориентированное программирование (AOP с использованием AspectJ)
- генерация страниц на сервере (средствами Thymeleaf)
- генерация страниц на клиенте (AJAX)
- объектно-реляционные отображения (реализация JPA с помощью ORM Hibernate)
- архитектура, управляемая событиями (EDA и ее реализация с помощью Kafka)
- Облачная архитектура
- Контейнерное развертывание

Учебные материалы, в том числе видео лекций, доступны онлайн по адресу

http://stepanoff.info/trsis/index.html

Репозиторий с примерами представлен по адресу

https://github.com/wildpierre/trsissamples

График выполнения лабораторных работ

Максимальное количество баллов за лабораторные работы – 72.

Для допуска к экзамену либо зачету требуется сдать ВСЕ лабораторные работы.

В таблице представлен максимальный рейтинг за лабораторную работу в зависимости от недели сдачи

	Лаб. 1	Лаб. 2	Лаб. 3	Лаб. 4	Лаб. 5	Лаб. 6	Лаб. 7	Лаб. 8
Нед. 1	9	9	9	9	9	9	9	9
Нед. 2	9	9	9	9	9	9	9	9
Нед. 3	8	9	9	9	9	9	9	9
Нед. 4	8	9	9	9	9	9	9	9
Нед. 5	7	8	9	9	9	9	9	9
Нед. 6	7	8	9	9	9	9	9	9
Нед. 7	6	7	8	9	9	9	9	9
Нед. 8	6	7	8	9	9	9	9	9
Нед. 9	5	6	7	8	9	9	9	9
Нед. 10	5	6	7	8	9	9	9	9
Нед. 11	4	5	6	7	8	9	9	9
Нед. 12	4	5	6	7	8	9	9	9
Нед. 13	3	4	5	6	7	8	9	9
Нед. 14	3	4	5	6	7	8	9	9
Нед. 15	2	3	4	5	6	7	8	9
Нед. 16	2	3	4	5	6	7	8	9
Нед. 17	1	2	3	4	5	6	7	8
Нед. 18	1	2	3	4	5	6	7	8

Pavel A. Stepanov, pavel@stepanoff.info © 2018 -2023
Кафедра компьютерных технологий и программной инженерии (№43) СПб ГУАП

Примерные варианты заданий

1. Книжный магазин либо библиотека
2. Поликлиника (запись на прием к врачу)
3. Расписание занятий в институте
4. Расписание поездов, самолетов, кораблей
5. Планирование покупок в магазине
6. Учет оценок студентов за лабораторные работы
7. Ведение списка литературы согласно последнему актуальному ГОСТу
8. Учет трат в бюджете семьи
9. Складской учет
10. Магазин электроники
11. Спортивные соревнования
12. Продажа автомобилей
13. Торговля акциями на бирже
14. Сдача недвижимости в аренду
15. Коллекционирование (нумизматика, филателия и пр).
16. Система безопасности предприятия (помещения, люди, права на вход)
17. Учет конфигурации сетевого оборудования сети предприятия
18. Учет показателей потребления электроэнергии и водоснабжения
19. База данных с рецептами блюд
20. База данных предметов живописи, скульптуры и пр. (музей)
21. Магазин компьютерных игр
22. Магазин музыкальных композиций
23. Учет содержимого холодильника (со сроками годности)
24. Учет медицинских лабораторных показателей пациента поликлиники
25. Ведение списка группы (ФИО, вариант задания, число сданных лабораторных, рейтинг) с учетом постоянно меняющегося количества студентов

Pavel A. Stepanov, pavel@stepanoff.info © 2018 -2023
Кафедра компьютерных технологий и программной инженерии (№43) СПб ГУАП

Технические требования

Рекомендуется разрабатывать программное обеспечение на платформе Java SE 17 https://adoptium.net/ и Spring 4. Spring рекомендуется использовать в варианте spring-boot.

В качестве базы данных рекомендуется использовать СУБД, не требующее установки, как например Derby или H2.

В качестве билд-системы рекомендуется использовать Maven 2 https://maven.apache.org/

В качестве сервера приложений рекомендуется использовать встроенный в приложение tomcat (автоматически загружается и развертывается maven/spring-boot).

Выполнение вышеуказанных рекомендаций позволит сократить набор инсталлируемых приложений до только платформы Java SE, а набор загружаемых библиотек только до Maven 2. Все остальные библиотеки и ресурсы Maven загрузит и установит самостоятельно.

В качестве IDE рекомендуется использовать IntelliJ Idea, Eclipse или Netbeans

Требования к отчету

Отчет должен содержать

- Титульный лист
- Текст и вариант задания
- Описание разрабатываемого продукта
- Текст основных фрагментов кода
- Отчет должен быть отправлен в личный кабинет студента через систему документооборота ГУАП
- Все артефакты лабораторной работы должны быть доступны в оригинальном (не напечатанном) виде, на случай если преподавателю потребуется их скомпилировать и выполнить.

Pavel A. Stepanov, pavel@stepanoff.info © 2018 -2023
Кафедра компьютерных технологий и программной инженерии (№43) СПб ГУАП

Конфигурирование рабочего места

Требования к рабочему месту

На компьютере пользователя должна быть установлена операционная система, поддерживаемая OpenJDK Java 17 (Windows, Red Hat Linux, Ubuntu, MacOS). Полный список поддерживаемых платформ можно получить на сайте https://adoptium.net/

На компьютере должен присутствовать интернет. Предполагаемый объем трафика, необходимого для выполнения лабораторной работы – несколько гигабайт.

Внимание. Интернет необходим только во время разработки. Конечный результат можно демонстрировать без интернета (при условии соблюдения двух условий – используется spring-boot и встроенный tomcat).

Установка Java

Установите последнюю версию Java 17 (Temurin) с сайта https://adoptium.net/

Обратите внимание, что это должен быть Java Development Kit (JDK), в не Java Runtime Environment (JRE)

Установка IDE

Установите одну из следующих IDE

IntelliJ Idea Community Edition https://www.jetbrains.com/idea/

Netbeans https://netbeans.org/

Eclipse http://www.eclipse.org/downloads/

Краткое описание системы сборки Maven

Внимание. Все, кому больше нравится Gradle, могут использовать Gradle. Все примеры к курсу идут с системой сборки Maven, но ничто не мешает их собирать Gradle.

Maven https://maven.apache.org является системой сборки программных продуктов с поддержкой жизненного цикла. Ее преимуществом является наличие центрального репозитория, содержащего официально выпущенные артефакты различных проектов. В процессе работы maven загружает актуальные версии библиотек в каталог ~/.m2 на Unix/Linux

или %HOMEPATH%/.m2 на Windows и в дальнейшем компонует из них Ваш проект. Поэтому данный каталог может достигать достаточно большого размера.

Проект на базе maven состоит из pom.xml файла (называемого также pom-файлом) – дескриптора проекта и каталога src с кодом. В каталоге src присутствуют два подкаталога – main и test. Первом хранится код продукта, а во втором – тестов.

В процессе сборки могут появляться другие каталоги, в частности, каталог target, который хранит артефакты сборки.

Вызов системы сборки maven может осуществляться как через ide (обычно в этом случае детали скрыты от пользователя), так и непосредственно командой “mvn цель” в каталоге с pom-файлом. Обычно используются следующие основные цели:

Цель **compile** – компиляция проекта

Цель **package** - компиляция проекта и сборка дистрибутива

Цель **install** – компиляция проекта, сборка дистрибутива и установка его в локальный репозиторий (чтобы другие проекты могли его использовать).

См. Также https://maven.apache.org/guides/introduction/introduction-to-the-lifecycle.html

Кроме того, дополнительные используемые фреймворки могут добавлять свои цели. В частности, запуск проекта spring-boot осуществляется целью **spring-boot:run** . Некоторые IDE скрывают эти цели от пользователя и преобразуют в обычные команды (построить, запустить и т.п.).

Лабораторная работа №1. Разработка простого серверного приложения с использованием сервлетов и фильтров

В данном разделе будет рассмотрена разработка простого веб-приложения, публикующего себя на веб-сервере.

Протокол http.

Для понимания того, как устроено веб-приложение, необходимо прежде всего понять, как устроен протокол http. Http (hypertext transfer protocol) является текстовым протоколом прикладного уровня, выполненным на основе транспортного протокола tcp. Типовое общение между клиентом и сервером состоит из выполнения запроса (Request) и получения ответа (Response).

Запрос состоит из нескольких частей:

- Метода запроса
- URI – идентификатора запрашиваемого ресурса
- Версии протокола
- Заголовков запроса
- Тела запроса

Ответ состоит из

- Кода ответа
- Тела ответа
- Заголовков ответа

Рассмотрим составляющие протокола подробнее. URI представляет собой адрес ресурса, которому отправляется запрос. Он формируется из протокола, имени сервера, номера порта и собственно пути, например, http://localhost:8080/mypp/myresource представляет собой получение ресурса /myapp/myresource с сервера localhost через порт 8080 по протоколу http. Данный путь является абстракцией и может обрабатываться веб-сервером по своему усмотрению, т.е. сервер решает, какой именно код обработает запрос по данному пути, на основании своей конфигурации, описывающей правила такого сопоставления. Кроме того,

URL может содержать список параметров в форме пар ключ=значение, разделенные амперсандом. Список параметров отделяется от основного пути знаком вопроса, например, http://localhost:8080/mypp/myresource?id=5

Метод запроса представляет конкретное действие, которое необходимо выполнить. Существует несколько стандартных методов, кроме того, ряд серверов позволяют определять собственные. Каждый метод имеет определенные ограничения и семантику, которая является не обязательной, но желательной.

Таблица 1. Виды стандартных методов HTTP

Метод	Стандартная семантика	Запрос имеет тело	Ответ имеет тело	Идемпотентность*	Кешируется
GET	Получить ресурс. Параметры обычно передаются через URL и их размер существенно ограничен.	Опционально	Да	Да	Да
POST	Создать ресурс. Параметры обычно передаются через тело.	Да	Да	Нет	Да
HEAD	Получить заголовки ответа GET	Нет	Нет	Да	Да
PUT	Изменить ресурс	Да	Да	Да	Нет
DELETE	Удалить ресурс	Нет	Да	Да	Нет
CONNECT	Создать тоннель	Да	Да	Нет	Нет
OPTIONS	Получить список поддерживаемых методов	Опционально	Да	Да	Нет
TRACE	Получить эхо запроса (чтобы проверить не вносят ли промежуточные сервера в него изменения)	Нет	Да	Да	Нет
PATCH	Частичная модификация ресурса	да	да	Нет	Нет

- Свойство идемпотентности означает, что несколько одинаковых запросов в рамках одной сессии приведут к тому же результату, что и один запрос. Например, если мы удаляем запись в таблице по ключу, то эту операцию можно повторить любое количество раз – эффект будет иметь только первая из них. Идемпотентность является

важнейшим свойством в нагруженных системах, так как в целом ряде случаев, в основном при сценариях обработки отказов, позволяет не задумываться о том, был ли вызов уже выполнен, а просто его повторить.

Обычно сервер обрабатывает, как минимум, методы GET и POST. Пре реализации REST API также обычно используются PUT, PATCH и DELETE. Принципиальная разница между PUT и PATCH заключается в том, что PUT полностью модифицирует ресурс, заменяя один другим, а PATCH меняет содержимое ресурса (например, если ресурс – запись о человеке, то PUT меняет одного человека на другого, а PATCH может поменять ему, например, только фамилию).

Заголовки запроса – пары вида ключ-значение. Они используются, чтобы передать серверу сопроводительную информацию. Таким образом можно указывать данные авторизации, информацию о формате запроса и тому подобную сервисную информацию. Можно также передавать серверу любые произвольные заголовки (предполагая, что пользовательский код на сервере сможет их обработать).

Наконец, тело запроса представляет ту информацию, которая отправляется на сервер. Запрос может и не иметь тела, если вся необходимая информация сосредоточена в заголовках и URL (типично для методов GET и DELETE).

Заголовки и тело ответа сервера принципиально не отличаются от заголовков и тела запроса. Важным отличием является код ответа. Код представляет собой трехзначное число. Различают пять групп кодов. Коды, начинающиеся на 1, сообщают о том, что обработка запроса еще не завершена. Коды, начинающиеся с 2, представляют нормальное завершение запроса. Коды, начинающиеся с 3, представляют ответ сервера при перенаправлении на другой ресурс. Коды, начинающиеся на 4, представляют сообщения об ошибках в запросе (синтаксических или семантических). Наконец, коды, начинающиеся на 5, представляют ошибки, произошедшие на сервере в процессе обработки запроса. Соответственно, запрос, успешно завершивший свое выполнение, обычно возвращает один из следующих кодов – 200 (OK), 201 (создано) или 202 (принято), реже 104 (успешно, но нет данных для возврата). Запрос, завершившийся с ошибкой по вине клиента, обычно возвращает 400 (неверный формат), 401 (не предоставлены

данные авторизации), 403 (данные авторизации предоставлены, но не дают доступа к ресурсу) 404 (ресурс не найден) и 409 (конфликт, текущее состояние системы не позволяет выполнить запрос). Статус 409 часто используют для индикации любых логических ошибок работы с ресурсами, наряду со статусом 422 (смысловая ошибка). Наконец, если на сервере произошла необработанная ошибка он отвечает статусом 500 (внутренняя ошибка). Важно также понимать значение статусов 502 (сервер работает как прокси и получил ошибочный ответ от следующего за ним сервера) и 504 (то же самое, но следующий сервер просто не ответил). Наконец, ошибка 503 говорит о том, что сервер недоступен про неизвестным, возможно, временным, причинам.

Полный перечень кодов статусов http запросов можно найти, в частности, по ссылке https://ru.wikipedia.org/wiki/%D0%A1%D0%BF%D0%B8%D1%81%D0%BE%D0%BA_%D0%BA%D0%BE%D0%B4%D0%BE%D0%B2_%D1%81%D0%BE%D1%81%D1%82%D0%BE%D1%8F%D0%BD%D0%B8%D1%8F_HTTP

Cookie.

Cookie (“куки”, буквально “печеньки”) – именованные фрагменты данных, которые сервер может хранить на клиентской машине и по мере необходимости перечитывать. Они специфичны для пользователя и для сайта и могут использоваться для решения таких задач, как запоминание предпочтений пользователя или информации о том, что пользователь залогинен. По ряду причин куки считаются небезопасными (скрытая передача пользовательских данных, возможность кражи куки из файловой системы злоумышленником и т.п.) и могут блокироваться политиками безопасности клиентского браузера, в этом случае их использование становится невозможным. Однако для нечувствительной информации использование кук существенно упрощает работу с клиентом.

Следующий демонстрационный фрагмент кода (с использованием библиотеки OkHttpClient) позволяет читать куки, которые присылает сервер:

```
public class Main {
        public static void main(String... args) throws IOException {
                class MyCookieJar implements CookieJar {
```

```
                private List<Cookie> cookies;
                public List<Cookie> getCookies() {
                        return cookies;
                }

                @Override
                public void saveFromResponse(HttpUrl url, List<Cookie> cookies) {
                        this.cookies = cookies;
                }

                @Override
                public List<Cookie> loadForRequest(HttpUrl url) {
                        if (cookies != null)
                                return cookies;
                        return new ArrayList<>();
                }
        }

        MyCookieJar cookieJar = new MyCookieJar();
        OkHttpClient.Builder builder = new OkHttpClient.Builder();
        builder.cookieJar(cookieJar);
        OkHttpClient client = builder.build();
        Request request = new Request.Builder().url("некоторый URL, который сюда надо поместить").get().build();
        Response response = client.newCall(request).execute();
        System.out.println("String list of cookies size:" + cookieJar.getCookies().size());
        for (Cookie cookie : cookieJar.getCookies()) {
                System.out.println("cookie " + cookie.name() + " has value " +
                cookie.value());
        }
    }
}
```

Http сессия.

Хотя правильным считается иметь сервер без состояния (причины чего будут рассмотрены позднее), иногда клиенту необходимо иметь данные, которые сохраняются между запросами.

Эта задача реализуется с помощью сессии, представляющей собой объект, содержащий данные, которые должны быть сохранены между запросами. Для того, чтобы каждый следующий запрос знал к какой сессии он принадлежит, сессии сопоставляется идентификатор и возникает задача его передачи с сервера на клиент и обратно, которая, в свою очередь, имеет две основных решения –через Cookie и через URL Rewriting.

В случае, если сессия реализуется через Cookie, то созданный на сервере объект ставится в соответствие некоторому идентификатору, который передается на клиент через Cookie. В дальнейшем клиент, чтобы обратиться к этой сессии, посылает идентификатор сессии обратно. Однако если клиентская система не разрешает Cookie, то используется URL Rewriting, то есть идентификатор сессии автоматически добавляется к каждому URL. Специальный механизм – HttpSession API – реализует эти сценарии автоматически и не требует от пользователя каких-либо специальных действий.

Архитектура простого веб-приложения.

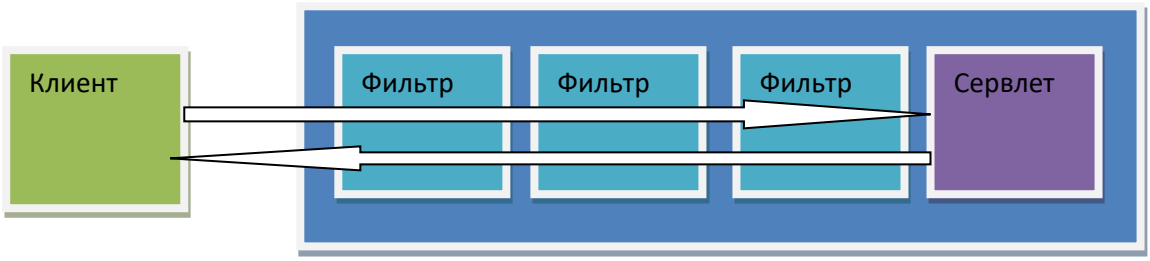
В современной IT – индустрии одним из наиболее важных типов приложений являются веб-приложения, то есть приложения, публикующие свой интерфейс через службу всемирной паутины. В данной лабораторной работе студент должен создать простое веб – приложение на языке Java, использующее фильтры и сервлеты. Простое приложение, построенное в такой архитектуре, представляет собой пользовательский код, выполненный в одном из двух форматов – сервлета или фильтра. В целом, следует иметь в виду, что разнообразные средства работы с шаблонами гипертекста или поддержки REST API в случае серверов, написанных на Java, в конечном итоге с очень большой вероятностью в своей основе имеют сервлеты и фильтры.

Фильтром называется пользовательский класс, реализующий интерфейс **Filter**. Важнейшим в этом интерфейсе является метод **doFilter,** который предназначен для реализации пользовательской логики при запросе к серверу. Фильтры организуются в последовательность, которая называется цепочкой фильтров и применяются по очереди. Цепочка фильтров формируется как приложением, так и наборами библиотек, которые это приложение использует и может использоваться для самых разнообразных целей, например,

таких как проверка прав доступа. Логика работы метода **doFilter** такова, что ему на вход передаются объекты, представляющие запрос к серверу и ответ сервера, а также специальный объект, представляющий следующий фильтр в цепочке. Фильтр должен принять решение – желает он сформировать ответ самостоятельно (обычно, выдать ошибку), либо передать эту задачу следующему в очереди фильтру. Кроме того, получив ответ от цепочки фильтров он также может произвести с ним какие-либо дополнительные действия, прежде чем возвратит управление предыдущему фильтру. Последний фильтр в цепочке передает управление сервлету, к которому был сделан вызов.

Сервлет – класс, реализующий интерфейс Servlet. Этот класс сопоставляется с определенным путем запроса на сервере и имеет в своем составе набор методов, которые вызываются в зависимости от http метода запроса. Наиболее обычным является реализация методов **doGet** и **doPost** для http методов **get** и **post** соответственно. Наиболее заметным функциональным отличием фильтра от сервлета является то, что фильтр не контролирует http метод, который был использован для его вызова, зато может применяться к путям на сервере, которые соответствуют определенному шаблону. Сервлет сопоставляется только с конкретным путем и может обрабатывать только те http методы, для которых у него есть реализация.

В общем случае запрос клиента по протоколу http проходит через цепочку фильтров и доходит до сервлета. Сервлет генерирует ответ и посылает его обратно клиенту. В ряде случаев один из фильтров может принять решение о том, что запрос не должен быть пропущен дальше и сгенерировать ответ самостоятельно (обычно это происходит в случае ошибки).



Поясним вышесказанное на следующем примере. Пусть на сервере имеются два ресурса, находящиеся по путям /public/data и /secure/data . Предполагается, что первый доступен без авторизации, а второй – только с авторизацией. Тогда можно написать фильтр, который будет

применяться ко всем путям, начинающимся на /secure , и проверять наличие у пользователя прав доступа к такому ресурсу. Сами же ресурсы будут обрабатываться двумя сервлетами, сопоставленными с путями /public/data и /secure/data соответственно.

Структура класса сервлета.

Сервлет – класс, представляющий http-ресурс. Он доступен по определенному адресу, который в примере определяется верез аннотации. Необходимо иметь в виду, что все запросы по этому адресу обрабатываются одним и тем же экземпляром класса сервлета, поэтому он не должен использовать потоконебезопасные ресурсы. Базовая структура сервлета следующая:

```
@WebServlet(name = "<имя сервлета>", urlPatterns = {"<url по которому доступен
сервлет>"})
public class ExampleServlet extends HttpServlet {
        @Override
        protected void doGet(HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException {
                //код, обрабатывающий HTTP метод GET
        }

        @Override
        protected void doPost(HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException {
                //код, обрабатывающий HTTP метод POST
        }
}
```

В данном примере реализован сервлет, реализующий наиболее часто встречающиеся http методы GET и POST, но можно реализовывать и другие методы http, используя соответствующие члены класса – doPut, doDelete, doHead, doOptions, doTrace. Также можно переопределить обобщенный метод service, в котором можно самостоятельно определить использованный http метод и его обработать.

Типовая структура кода обрабатывающего метода следующая – он берет параметры из объекта request и формирует данные объекта response (ответа сервера). Для формирования тела ответа сервера у объекта response вызывается метод getWriter или getOutputStream в зависимости от того, является ответ текстовым или бинарным. Особенность этого метода заключается в том, что вызвать его можно только один раз, поэтому если какой-либо фильтр предполагает менять ответ сервлета на лету, он должен передать ему какой-то другой поток вывода, а потом самостоятельно заполнить оригинальный поток вывода сервера.

Структура класса фильтра.

Фильтр – класс, содержащий код, выполняемый до и после вызова сервлетов (и некоторых других фильтров). В зависимости от положения фильтра в цепочке фильтров до и после него могут выполняться другие фильтры. Типовое применения фильтра – аудит безопасности, логирование либо замена ответа сервлета на лету.

Структура класса следующая:

```
@WebFilter("/example")
public class ExampleFilter implements Filter {
        public void init(FilterConfig filterConfig) throws ServletException  {  }
        public void destroy()  {  }

        @Override
        public void doFilter( ServletRequest request,  ServletResponse response,  FilterChain
        chain) throws IOException, ServletException {
                // Сюда вставляется пользовательский код, выполняемый перед
                остальными фильтрами и сервлетом
                chain.doFilter(request, newResponse);
                // Сюда вставляется пользовательский код, выполняемый после
                остальных фильтров и сервлета
        }
}
```

В данном примере реализован фильтр, который не делает ничего кроме передачи управления далее по цепочке фильтров.

Задание на лабораторную работу.

Выполните следующие задачи.

1. В соответствии со своим вариантом разработайте набор из минимум двух экранных форм приложения
2. Соберите проект веб-приложения (war) на Maven (можно без использования Spring)
3. реализуйте формы средствами сервлетов. Проект должен как минимум содержать формы просмотра, добавления и удаления данных. **Не используйте технологию JSP (Java Server Pages)**
4. Аргументируйте почему были выбраны те или иные запросы HTTP.
5. Использовать базу данных можно, но не обязательно

Pavel A. Stepanov, pavel@stepanoff.info © 2018 -2023
Кафедра компьютерных технологий и программной инженерии (№43) СПб ГУАП

Лабораторная работа №2. Разработка ресурса REST/JSON сервиса

Предпосылки появления Web API

При разработке приложений крайне неудобно возвращать всегда непосредственно HTML страницы. Это связано с тем, что таким образом мы объединяем логику доступа к данным и логику представлений в одном месте. Недостатком такого объединения может быть, например, то, что при появлении нового клиента доступ к данным необходимо реализовывать и в нем. Эта проблема может быть решена путем вынесения логики доступа в отдельную библиотеку, однако возникающие сложности по управлению конфигурациями всех программных продуктов являются причиной появления систем, в которых возможно разделение кода на отдельные компоненты, исполняемые на различных машинах, и обмен данными по четко специфицированным протоколам. Первоначальное развитие получили технологии, основанные на бинарных протоколах, такие как CORBA/IIOP и DCOM/ORPC, через которые мультимашинные приложения могли обмениваться информацией. В частности, технология CORBA в своей основе содержит ORB (Object Request Broker) – унифицированную шину данных, к которой можно подключиться и через которую передаются и получаются данные. Компонент, желающий выполнять передачу по шине, должен знать контекст адресата своего сообщения и выполнять конвертацию вызовов и данных в бинарный формат (процесс называется маршаллингом) и из бинарного формата (демаршаллинг) в соответствии с правилами, определенными для языка, на котором этот компонент написан. В результате с помощью технологии CORBA можно интегрировать коды на десятках различных языков.

Тем не менее такие подходы оказались неудобными (прежде всего из-за высокой сложности) и, как следствие, появилось понятие Web API - интерфейса программирования, предоставляемого через WEB. Обычно такой API получает данные и возвращает их в виде одного из стандартных форматов, наиболее популярными из которых являются XML и JSON (Java Script Object Notation).

SOAP

SOAP является технологией создания интерфейса приложения, основанной на XML. В основе подхода лежит идея о том, что веб-компонент публикует описание своего интерфейса на

специальном языке WSDL (т.е. XML с определенной схемой), а все его клиенты могут этот дескриптор прочитать и по нему определить правила вызова интерфейсных методов. Полностью этот язык описывается в документе https://www.w3.org/TR/wsdl/ . Пример дескриптора можно посмотреть здесь: https://www.w3.org/2001/04/wsws-proceedings/uche/wsdl.html

Получив ссылку на интерфейс, клиент SOAP-приложения может осуществлять вызов. При использовании SOAP все данные и вызовы преобразуются в специальные XML-посылки, называемые SOAP-конвертами (envelop), которые передаются между клиентом и сервером обычно через протокол HTTP или HTTPS.

JAVA-реализация SOAP называется JAX-WS. В виду того, что создавать дескрипторы непосредственно руками крайне неудобно, обычно разработка программного обеспечения на основе JAX-WS заключается в использовании разнообразных мастеров, автоматически генерирующих и обновляющих дескрипторы. Это делает код, связанный с обработкой дескрипторов, тяжело читаемым разработчиком. Еще одним важным недостатком SOAP является использование такой тяжеловесной технологии, как XML – маршалинг и демаршалинг для XML являтся очень ресурсоемкой операцией даже с использованием легковесных парсеров типа SAX. Поэтому при прочих равных условиях SOAP проигрывает в производительности более легким решениям.

JSON

В настоящее время лидирующим решением среди технологий организации интерфейсов является JSON – Java Script Object Notation. В этой технологии на клиент передается сериализованный в текст объект Java Script. При этом, хотя технически возможно передать объект вместе с кодом, обычно рассматриваются только поля данных.

Для того, чтобы получить представление о том, как работает JSON, рассмотрим пример Java класса и соответствующий ему JSON:

Простой Java класс (POJO, Plain Old Java Object)

```
@Data
```

```
public class CustomData {
        String name;
        Map<String, String> attributes;
        Set<String> array;
}
```

И возможный JSON:

{"name": "Some Name", "attributes": {"someAttribute": "some value", "anotherAttribute ": "another value"}, "array": ["value1", "value2", "value2"]}

Как можно видеть, конечная структура JSON зависит от данных. В частности, если бы элементы массива или отображения имели сложную структуру (были массивами или объектами), то они также были бы преобразованы в JSON. Преобразование POJO в JSON и обратно происходит автоматически такими библиотеками, как Jackson (считается наиболее производительной) или Gson; однако очевидно, что обратное преобразование, т.е. из JSON в POJO не может быть выполнено единообразно (например, когда мы преобразуем поле attributes в примере выше мы не можем сказать, является результат типом Map или объектом с двумя полями, а array может быть преобразован как в Set, так и в массив). Поэтому для того, чтобы выполнить обратное преобразование, конвертерам требуется прототип того POJO объекта, в который мы собераемся демаршалить JSON.

Тем не менее, данный метод организации интерфейса имеет существенное преимущество – JavaScript может быть использован как на сервере, так и на клиенте, поэтому API, построенный на JSON, может быть напрямую использован клиентскими сценариями в браузере. Библиотеки AJAX (Jquery, AngularJS) могут получать результаты вызовов и анализировать их структуру естественным для языка браузерных сценариев образом.

RESTful API

Необходимо четко понимать, что не любой API, основанный на JSON, является RESTful. REST – это концептуальный подход; технически те же самые задачи могут быть решены JSON

сервисами, которые не соблюдают некоторые или все концепции REST. В основе REST лежит ряд концепций, которым необходимо удовлетворять, а именно:

1. Модель является клиент-серверной, т.е. сервер предоставляет REST интерфейс, через который клиент осуществляет вызов (по протоколу HTTP/HTTPS);
2. Сервер не имеет информации о состоянии клиента (отсутствует сессия) – это значительно упрощает масштабирование (при этом сервер может иметь внутреннее, не публичное, состояние, например, какие-либо кеши данных);
3. Сервер может осуществлять кеширование данных (строго в соответствии с таблицей 1);
4. Клиентской системе не известно что именно находится а URL – один сервер или несколько серверов, возможно, обьединенных в иерархии;
5. Клиент манипулирует ресурсом через его представление.

Следующие ограничения при создании REST-интерфейсов зачастую игнорируются:

6. Каждое сообщение должно содержать достаточно информации о том, как его обрабатывать (например, иметь заголовки, указывающие на допустимые операции и т.п.)
7. Сообщение должно содержать в себе свой описатель (в самом простом случае – ссылку, по которой доступен тот ресурс, который находится в этом сообщении). Это требование обозначается аббревиатурой HATEOAS.
8. Сервер может возвращать не только данные, но и код. Это последнее требование является опциональным.

Рассмотрим пример организации REST – интерфейса. Пусть имеется множество объектов *CustomData* из примера выше. Мы можем разместить их по адресу http://<server>/app/customdata/ . Тогда объект с конкретным именем может иметь URL http://<server>/app/customdata/<имя> или http://<server>/app/customdata/name/<имя>. Соответственно, выполнение метода GET с указанным URL будет приводить к чтению этого объекта, POST и PUT будут использоваться для создания и изменения объекта, а DELETE – его удаления. Начинающие разработчики испытывают желание решать эту задачу иначе,

например, удалять объект через URL вида http://<server>/app/customdata/<имя>/delete , что является технически возможным, но нарушающим правила REST, так как этот URL не является идентификатором ресурса.

Инструменты разработчика

Вероятно, наиболее популярным средством работы с запросами REST является postman https://www.getpostman.com/ . Это приложение позволяет, в частности, выполнять запросы с указанием тела запроса, метода и заголовков и получать назад ответы в разобранном виде.

Реализация REST средствами Spring-boot

В составе библиотеки spring-boot-starter-web находится мощное средство для организации REST. Для того, чтобы объявить бин, реализующий рест интерфейс, используется аннотация @RestController. Далее этому контроллеру можно указать базовый адрес, относительно которого будут строиться адреса его методов; для этого используется аннотация @RequestMapping(<адрес>). Сконфигурированный таким образом контроллер может декларировать методы, отображающиеся в рест – интерфейс. Для этого используются следующие основные аннотации (на методах):

@RequestMapping(value = "<адрес>", method =<метод>), где метод принимает значение константы класса RequestMethod – объявляет адрес и http метод доступа к REST – ресурсу;

@RequestParam("<имя параметра>") позволяет аннотировать параметр метода, после чего в него будет автоматически передан соответствующий параметр формы;

@RequestBody позволяет аннотировать параметр метода, после чего в него будет автоматически передан JSON (если мы используем его, а не форму, для передачи параметров). В этом случае Объект будет построен из JSON стандартным мапером, обычно Jackson.

Рассмотрим некоторые примеры. Следующий контроллер опрееляет рест-интерфейс

```
@RestController
@RequestMapping("/public/rest/resource")
public class SampleRestController {
```

```
        @RequestMapping(value = "/resource1", method = RequestMethod.GET)
        public Resource1 getResource1() { /*код*/  }
    }
```

Определяет сервис /public/rest/resource/resource1 доступный методом GET без параметра. Если мы хотим добавить параметр то мы можем действовать двумя способами – передавать его в списке параметров URL /public/rest/resource/resource1?id=1 или через основную часть URL /public/rest/resource/resource1/1 . В первом случае (который должен использоваться только для http метода GET) определение будет выглядеть как

```
@RestController
@RequestMapping("/public/rest/resource")
public class SampleRestController {
        @RequestMapping(value = "/resource1/{id}", method = RequestMethod.GET)
        public Resource1 getResource1(@RequestParam("id" Long id)) { /*код*/  }
}
```

Во втором случае определение будет выглядеть так:

```
@RestController
@RequestMapping("/public/rest/resource")
public class SampleRestController {
        @RequestMapping(value = "/resource1/{id}", method = RequestMethod.GET)
        public Resource1 getResource1(@RequestParam("id" Long id)) { /*код*/  }
}
```

Заметим, что возвращаемый объект типа Resource1 будет преобразован в JSON автоматически, с возвращением кода 200. Если нам необходимо вернуть не только данные, но еще заголовки и код статуса операции, можно воспользоваться расширенным определением:

public ResponseEntity< Resource1> getResource1(**@RequestParam**("id" Long id)) { /*код*/ }

В такой версии определения метода можно вернуть данные вместе с заголовками и кодом; например, следующим образом:

return ResponseEntity.status(HttpStatus.CREATED).body(<объект типа Resource1>);

В качестве кода возврата будет использован HttpStatus.CREATED , т.е. 201. Отметим, что с точки зрения семантики REST использование этого кода должно идти совместно с RequestMethod.POST.

Развитие REST – Query Data API

Существуют подходы, основанные REST, и предназначенные для получения данных от сервера по определенным запросам, сформулированным клиентом. Прежде всего, это GraphQL https://graphql.org/learn/ . В технологии GraphQL на сервер посылается JSON с запросом в определенном формате, а назад приходит ответ, опять таки, в определенном формате. Этот подход, с одной стороны, гибче REST и позволяет экономить на передаче информации, которая не требуется клиенту, с другой стороны, перекладывает часть логики приложения на клиент, что является определенным недостатком.

Другим развивающимся подходом является OData https://www.odata.org/ . Подход представляет собой формальный язык запросов к сервису, выполненный в стандарте REST и обладающий высокой гибкостью.

ORDS (Oracle Rest Data Service) https://www.oracle.com/database/technologies/appdev/rest.html является инструментом, автоматически отображающим определенным образом сформированные http –запросы на базы данных.

Сравнительный анализ этих технологий можно прочитать по ссылке https://www.progress.com/blogs/rest-api-industry-debate-odata-vs-graphql-vs-ords (англ.).

Формальные описания интерфейсов

В целом ряде случаев требуется описать интерфейс приложения раньше, нежели написать само приложение. Это может быть необходимо в целом ряде случаев, например, чтобы закрепить контракт на API при разработке нескольких микросервисов разными командами одновременно, генерации клиента для b2b взаимодействия или для конфигурирования API GateWay.

На момент написания этого пособия отраслевым стандартом является определение интерфейсов через OpenAPI v3. Составить представление о нем можно на сайте https://editor.swagger.io/

Например, простой API двумя методами post и get, доступный по пути /schools, может быть описан так (расстояние текста от начала строки имеет важное значение):

```
openapi: 3.0.1
info:
  title: Университет
  description: Пример для студентов
  termsOfService: https://www.apache.org/licenses/LICENSE-2.0
  contact:
    email: pavel@stepanoff.info
  license:
    name: Лицензия Apache 2.0
    url: http://www.apache.org/licenses/LICENSE-2.0.html
  version: 1.0.0
paths:
  /school:
    post:
      tags:
        - api факультетов
```

```
    summary: Добавить новый факультет
    operationId: addSchool
    requestBody:
      description: Описание метода
      content:
        application/json:
          schema:
            $ref: '#/components/schemas/School'
      required: true
    responses:
      '400':
        description: Неверный формат ввода
        content: {}
      '201':
        description: Успешная операция
        content:
          application/json:
            schema:
              type: array
              items:
                $ref: '#/components/schemas/School'
  get:
    tags:
      - api факультетов
    summary: Найти факультет по идентификатору
    description: Описание метода
    operationId: findSchoolById
    parameters:
```

```
        - name: id
          in: query
          description: идентификатор факультета
          required: true
          schema:
            type: integer

      responses:
        '200':
          description: успешная операция
          content:
            application/json:
              schema:
                type: array
                items:
                  $ref: '#/components/schemas/School'
        '404':
          description: Факультет не найден
          content: {}
components:
  schemas:
    School:
      type: object
      description: объект, представляющий данные о факультете
      properties:
        id:
          type: integer
          format: int64
```

```
      description: идентификатор
    number:
      type: integer
      format: int64
      description: номер факультета
    name:
      type: string
      description: Название факультета
```

Как можно видеть, спецификация весьма многословная и слабо пригодна для распечатки в виде текста, однако по ней можно генерировать сервера и клиенты на различных языках.

Задание на лабораторную работу.

1. Определите перечень Rest-сервисов, выполняющих те же действия, что и в лабораторной работе 1. Внимательно отнеситесь к вопросу какой HTTP метод использует тот или иной сервис и какие коды HTTP он может возвращать.
2. Опишите Ваш API с помощью OpenAPI v3. **Приведите Ваш yaml файл в отчете**.
3. API должен содержать все четыре метода CRUD – добавление, удаление, изменение и чтение. Следите за тем, чтобы Ваш API оперировал только JSON представлениями и не возвращал в качестве результатов строки с сообщениями.
4. Средствами swagger сгенерируйте сервер spring (необязательно).
5. Если Вы выполнили пункт 4, возьмите REST контроллер из сгенерированного сервера, вставьте в свое приложение (не забыв поправить имя пакета). В противном случае напишите контроллер самостоятельно.
6. Реализуйте приложение. Swagger UI должен быть доступен через web (по умолчанию путь - http://localhost:8080/swagger-ui/index.html). Swagger UI должен быть обязательно включен.

Лабораторная работа №3. Разработка простого AJAX приложения Spring

Архитектура клиентского представления.

При программировании для сети Интернет приложение приобретает другую структуру, нежели при программировании для десктопов. Само приложение состоит из двух частей – серверной части, или бэкенда (backend), исполняемого где-то в интернете и клиентской части или фронтенда (frontend), исполняемого на клиентской машине в браузере. При этом фронтенд генерируется опять-таки на сервере и запрашивает с сервера ресурсы, необходимые для своей работы. Одновременно браузер скрывает от фронтенда технические детали, связанные с доступом к серверной части, а также следит за безопасностью.

Браузер обычно выполняет код двух видов – описания страниц на HTML и сценариев на JavaScript. Первые приложения таких типов имели статические клиентские страницы – при работе с ними было необходимо отправить на сервер запрос и получить в качестве ответа новую страницу. Такой подход иногда используется и сегодня (обычно для простых приложений с “легкими” страницами). Тем не менее он неудобен тем, что пользователь, нажав на кнопку, вынужден ждать полной регенерации своего представления; вместо этого пользователю было бы удобно чтобы страница реагировала только в той ее части, которая является релевантной воздействию. Есть и другие недостатки, в частности, затруднено масштабирование, “тяжелые” страницы интенсивно потребляют сетевые ресурсы и т.п.

Для решения указанной выше проблемы разрабатывались различнве подходы – это и загрузка скриптов в скрытые фреймы, XML острова данных, интеграция браузерных расширений (Flash, Silverlight, Java plugin) и пр. В настоящее время со введением стандарта HTML5 доминирующим решением является технология AJAX.

Технология Ajax.

Технология AJAX, или “Asynchronous JavaScript and XML” была предложена в середине двухтысячных годов как обобщение всех имеющихся на тот момент технологий асинхронного доступа к ресурсам сервера. Ключевой идеей технологии было использование в браузере интерфейса XHR (XMLHttpRequest) для получения ресурсов сервера, которые могут

предоставляться в дружественном браузеру формате (JSON, JavaScript); в результате сценарии браузера, написанные на JavaScript, могут обратиться к серверу, получить обьекты в формате JSON и динамически встроить результат в текущий документ браузера, используя DOM (Document Object Model). Другим вариантом действий является непосредственная доставка с сервера исполняемого кода на JavaScript.

В настоящее время имеется множество популярных библиотек, обрабатывающих пользовательский интерфейс средствами AJAX, а именно JQuery, Angular JS, React JS и другие.

Библиотека AngularJS.

AngularJS https://angularjs.org/ является библиотекой java script, использующей DOM для манипуляции документом на основании дополнительной разметки. Библиотека использует паттерн MVC, где в роли контроллера выступают коды на джаваскрипте, а в роли модели – элементы DOM.

Приложение AngularJS основывается на трех основных директивах. Директива **ng-app** определяет связь приложения с html страницей, директива **ng-model** связывает данные приложения с управляющими элементами на форме (изменение элемента приводит к изменению данных модели) и директива **ng-bind** связывает данные приложения с разметкой HTML (изменение модели приводит к изменению элемента). Последняя директива обычно записывается сокращенно- в виде двух фигурных скобок. Angular поддерживает использование для значений директивы **ng-bind** выражений на языке javascript. Кроме указанных, очень важной директивой является **ng-repeat**, которая используется для построения списков и таблиц.

Рассмотрим тривиальный пример, который использует angularJS для демонстрации своих базовых возможностей.

```
<!doctype html>
<html ng-app>
    <head>
```

```
        <script
        src="https://ajax.googleapis.com/ajax/libs/angularjs/1.
        7.5/angular.min.js"> </script>
    </head>
    <body>
        <div>
            <label>Name:</label>
            <input type="text" ng-model="yourName"
            placeholder="Enter a name here">
            <hr/>
            <h1>Hello {{yourName}}!</h1>
        </div>
    </body>
</html>
```

Страница помечена атрибутом ng-app, что информирует angular о необходимости ее обработки. Поле ввода помечено атрибутом ng-model, что информирует angular о необходимости установить листенер на изменение этого поля и запись его значения в переменную yourName. Наконец, чуть ниже в коде встречается инструкция {{yourName}}, что информирует angularJS о необходимости записать туда значение соответствующей переменной, которая, как мы помним, связана с полем ввода. В результате при изменении значения в поле ввода автоматически меняется текст в заголове <h1>.

Важное значение в AngularJS имеют объекты **$scope** и **$http**. Директива ng-controller определяет набор функций, которые отвечают за работу определенной формы, и **$scope** передается в каждую из них как параметр, указывающий на приложение, работающее с этим контроллером. Соответственно, в **$scope** можно определять глобальные переменные и т.п. Объект **$http** используется для целей получения REST ресурсов. Типовой вызов **$http** выглядит следующим образом (в данном случае для метода post):

```
$http.post(address).then($scope.successCallback, $scope.errorCallback);
```

В зависимости от того, увенчался метод успехом или нет, будет вызван либо метод $scope.successCallback либо метод $scope.errorCallback, в которые будет передан объект, представляющий ответ. Соответственно, именно в них должна находиться вся обработка,

связанная с AJAX. В случае успешного завершения запроса в поле **data** обьекта, представляющего ответ, будет передан JSON, то есть объект Java Script, который может быть разобран, связан с переменными модели и, таким образом, отображен на форме.

Задание на лабораторную работу.

1. Используйте в качестве основы API, разработанный в лабораторной работе номер 2.
2. Создайте одну или несколько форм и контроллеров для страниц приложения. Подключите библиотеку angularJS (желающие могут воспользоваться JQuery или иным аналогом).
3. Реализуйте задачи из лабораторной работы 1 средствами AJAX и REST-сервисов, разработанных в предыдущей лабораторной работе.

Лабораторная работа №4. Разработка формы логина

Основы авторизации и аутентификации. Виды аутентификации.

Несмотря на то, что существует определенная путаница в этих понятиях, авторизация и аутентификация – два разных процесса. Аутентификация – процесс определения того что пользователь является именно тем, кем он представляется. Авторизация – процесс определения того какие права имеет аутентифицированный пользователь.

Существует огромное количество моделей аутентификации, в частности,

1. HTTP BASIC
2. HTTP Digest
3. HTTP X.509 client certificate exchange (an IETF RFC-based standard)
4. LDAP
5. Kerberos
6. SSO
7. OpenID
8. OAuth
9. Form-based authentication

И многие другие. Ниже некоторые модели будут рассмотрены более подробно.

Виды атак на веб-приложение.

Веб-приложение обычно является приложением, распространяемым через незащищенную среду (интернет). Даже в случае внутренней сети предприятия или защищенных специализированных криптосетей всегда есть вероятность того, что среда передачи скомпрометирована. Поэтому, разрабатывая веб-приложение, необходимо уделить внимание следующим аспектам:

1. Защита данных при передаче (шифрование), ведущая к исключению атаки проксированием или путем перехвата пакетов данных снифферами (приложениями, прослушивающими сеть и перехватывающими пакеты, предназначенные другим хостам);

2. Защита от кросс-сайтовой подделки запроса (CSRF) и межсайтового скриптования (XSS);
3. Аутентификация (подтверждение личности) и авторизация (подтверждение прав) стандартными средствами и алгоритмами, для которых подтверждена безопасность;
4. Разбиение ресурсов приложения по зонам доступа, присвоение им требований ролей и назначение ролей пользователям в соответствии с принципом минимально необходимых прав.

Снифферы и атаки “Man-in-the-middle”.

Сниффер (анализатор траффика) – это программа, которая переводит сетевую карту в режим, в котором она анализирует все пакеты в своем сегменте сети, вне зависимости от их адресата. Такое программное обеспечение может применяться администраторами сетей для обнаружения неправильной маршрутизации трафика; к сожалению, оно же может применяться злоумышленниками для перехвата трафика пользователя. Существуют инструменты обнаружения снифферов, но в тот момент, когда системный администратор обнаружит вредоносное ПО, захват чужих данных уже произойдет. Поэтому критически важно, чтобы весь трафик, содержащий логины и пароли, был зашифрован (а также грамотно настроена маршрутизация трафика, максимально исключая его отправку в сегменты сети, которым он не предназначается).

Другой важной проблемой является злонамеренная конфигурация прокси сервера. В случае, если доступ к нему получил злоумышленник, то весь трафик пользователя автоматически становится ему доступен. Такая атака называется “человек посередине” или man-in-the-middle. К счастью, современные алгоритмы шифрования устойчивы к таким атакам, что не в последнюю очередь достигается тем, что трафик подписывается цифровой подписью, удостоверенной сертификатом, заверенным сертифицированным удостоверяющим центром. Преодоление такой защиты крайне сложная задача, включающая подмену служб DNS (чтобы она указывала на фальшивый удостоверяющий центр) или подмену предымпортированных в систему сертификатов, что, вообще говоря, практически невозможно без получения физического доступа к системе или сети. К сожалению, получение сертификата безопасности стоит заметных денег, поэтому в рамках лабораторной работы мы будем использовать

“самоподписанные” сертификаты, генерируемые программной keytool, входящей в состав платформы java. Такие сертификаты не могут быть проверены удостоверяющим центром и потому могут быть использованы только в целях разработки ПО. Например, такой сертификат может быть сгенерирован плагином maven:

```
<plugin>
    <groupId>org.codehaus.mojo</groupId>
    <artifactId>keytool-maven-plugin</artifactId>
    <version>1.5</version>
    <executions>
    <execution>
        <goals>
        <goal>clean </goal>
        <goal>generateKeyPair</goal>
        </goals>
        <phase>generate-resources</phase>
    </execution>
    </executions>
    <configuration>
    <alias>tomcat</alias>
    <storetype>PKCS12</storetype>
    <keyalg>RSA</keyalg>
    <keysize>2048</keysize>
    <keystore>${project.basedir}/target/keystore.p12</keystore>
    <dname>CN=stepanoff.info, OU=stepanoff.info, O=stepanoff.info, L=SPB, ST=SPB,
    C=NO</dname>
    <validity>3650</validity>
    <password>mypassword</password>
    <storepass>mypassword</storepass>
```

```
            <workingDirectory>${basedir}</workingDirectory>
            <storetype>pkcs12</storetype>
            </configuration>
        </plugin>
```

При попытке перехода на сайт, защищенный таким сертификатом, браузер будет выдавать предупреждение о невозможности его проверки. Так как мы знаем, что сами сгенерировали этот сертификат, мы можем считать переход на сайт **localhost** безопасным и пропустить предупреждение браузера; в большинстве случаев, встретив аналогичное сообщение в сети интернет, можно обоснованно предполагать, что сайт был взломан (либо, в ряде случаев, заблокирован Роскомнадзором).

Защита от атак XSS и CSRF.

XSS – Cross-Site Scripting, специальный вид атаки, при котором вредоносный код пытается загрузить в себя часть доверенного сайта и выполнить в нем действия от имени пользователя. Существует много различных вариантов этой атаки, начиная с загрузки доверенного сайта во фреймы и заканчивая попытками инъекции сценариев в поля ввода доверенного сайта, в надежде что эти сценарии будут выполнены и сумеют получить доверенный доступ к сценариям самого доверенного сайта.

Загрузка доверенного сайта полностью или частями во вредоносный сайт в основном блокируется современными браузерами на основании политики same-of-origin (единого происхождения), то есть требования, чтобы все ресурсы сайта приходили с одного доменного имени, протокола и порта. Иногда доверенный сайт использует технологию CORS (Cross-Origin Resource Sharing) позволяющую информировать браузер об исключениях в правиле same-of-origin. Использование этой технологии не рекомендуется с точки зрения безопасности; неправильное же ее употребление может привести к уязвимости сайта.

Инъекция кода javascript опасна тем, что злонамеренный код выполняется в контексте того же сайта, что и вызываемые им доверенные сценарии. Ключевым средством борьбы с инъекцией скриптов является правильная обработка полей ввода и преобразование всех специальных

символов в их символьные эквиваленты (< в &lt, > в &gt и тд). Подобные преобразования делают код неисполняемым. Тем не менее, все равно не рекомендуется вставлять потенциально небезопасные данные в страницу на сервере.

Еще одним типом атаки является CSRF – Cross-Site Request Forgery (межсайтовая подделка запроса). Эта атака происходит при попытке отправить форму в доверенный сайт, который может посчитать, что пользователь имеет cookie и потому залогинен, а следовательно запрос достоверен. Для борьбы с этим видом атак в Spring используется специальный токен, выдаваемый сервером, который необходимо указывать в параметрах запросов POST, PUT, PATCH и DELETE. Использование такого токена, естественно, возможно только при наличии сессии – в противном случае было бы невозможно установить, какой токен какой странице соответствует.

Хеширование паролей.

Одним из наиболее серьезных рисков при создании систем аутентификации является утечка паролей. Связано это с двумя аспектами – во-первых, злоумышленник, получивший доступ к базе, получает доступ к данным всех пользователей; во-вторых, пользователь, обладающий достаточно устойчивым паролем, имеет обыкновение использовать его на всех сайтах, соответственно, утечка пароля на одном сайте приводит к уязвимости его данных на всех остальных. Разработчики защиты могут частично побороть эту проблему за счет двухфакторной авторизации (т.е. в определенных или во всех случаях пользователь должен ввести код, пришедший ему на телефон), но двухфакторная авторизация используется далеко не всегда, а зачастую пользователи сами от нее отказываются, как от неудобной. Поэтому критически важно сделать так, чтобы доступ к базе не привел к немедленной утечке паролей.

Для того, чтобы избежать утечки, в базе хранят не пароли, а их хеши – то есть пароли, преобразованные специальной функцией (хэш-функцией) таким образом, чтобы обратное преобразование требовало огромных вычислительных ресурсов. Соответственно, для того, чтобы проверить, что пользователь ввел правильный пароль, сравниваются не сами пароли, а их хеши, и получив в свое распоряжение хэш восстановить их него пароль, не имея

суперкомпьютера и большого количества времени, невозможно. Spring рекомендует использовать для хеширования алгоритм BCrypt https://en.wikipedia.org/wiki/Bcrypt

Basic и Digest аутентификация.

Basic аутентификация является простейшей формой аутентификации, не требующей никаких специальных средств. В этой модели аутентификации клиент посылает серверу запрос с заголовком Authorization, в содержимом которого присутствует слово Basic и пара login:password закодированная в Base64. Заметим, что алгоритм Base64 является симметричным и потому абсолютно небезопасен. Сервер раскодирует пару login:password и, в зависимости от результатов проверки, авторизует или не авторизует пользователя. Другим вариантом передачи этой пары является прямое кодирование в URL, например https://login:password@адрессайта . Этот формат авторизации является устаревшим (RFC-3986).

Со своей стороны, сервер оперирует заголовками и кодом возврата. Если ресурс требует аутентификации, то сервер должен прислать статус 401 и заголовок WWW-Authenticate: Basic, в котором указывается Security Realm (абстракция, представляющая защитный механизм, которому известен этот пользователь).

Так как было бы очень неудобно каждый раз запрашивать пользователя о его логине и пароле, то браузеры их кешируют, причем конкретный алгоритм хеширования зависит от браузера. Это ставит проблему разлогинивания, которая решается различными техниками, связанными со сбросом закешированных данных пользователя.

Алгоритм Basic аутентификации считается слабозащищенным из-за передачи по сети пары логин : пароль в незашифрованном виде. Поэтому, как компромиссное решение, появилась аутентификация Digest (RFC-2069, RFC-2617), которая передает строку логин : realm : пароль, хешированную по умолчанию алгоритмом md5. Так как разобрать хэш-- функцию достаточно сложная вычислительная задача, Digest аутентификация сравнительно значительно надежнее, чем Basic, однако по прежнему уязвима к атакам типа “man-in-the-middle” (отметим, что

хеширование md5 также считается небезопасным, необходимо использовать, например, SHA-256).

Аутентификация через директории пользователей (Kerberos, LDAP).

В сложных корпоративных системах зачастую возникает проблема, связанная с тем, что одному и тому же пользователю надо назначать права во многих приложениях одновременно. В такой ситуации бывает полезно вынести центр аутентификации из приложения в стороннюю систему. Примерами таких решений могут быть Kerberos и LDAP (а также многие другие).

Kerberos является протоколом шифрования, при котором аутентификация возможна даже если между приложением и центром аутентификации лежит незащищенный канал. Такой протокол шифрования, например, используется службой Active Directory. Приложение и центр авторизации обмениваются данными о пользователе по специальному протоколу, используя симметричное шифрование Нидхема-Шредера.

LDAP (Light-Weight Directory Access Protocol, легковесный протокол каталога пользователей) является протоколом, часто используемым для централизованного хранения имен пользователей и паролей. Этот протокол не просто разрешает аутентифицировать пользователей, но еще и управлять ими, то есть добавлять, удалять и модифицировать записи.

Использование аутентификации через директории пользователей удобно в крупных компаниях, в которых необходимо иметь единое место ведения информации о пользователях. Кроме того, эта модель аутентификации значительно безопаснее, чем Basic и Digest, по многим причинам: сервер является внешним для приложения и может быть закрыт файрволлом, его безопасность подтверждена аналитиками, правила его безопасного использования четко определены и т.п.

Внешняя аутентификация через клиент (OpenID, OAuth).

В ряде случаев компании может быть неудобно поддерживать собственный сервер с аутентификационной информацией пользователей. Одной из причин этого является отсутствие специализированных персональных данных, отличающихся от других сайтов, а

также то, что новому пользователю может быть неудобно проходить процесс регистрации на десятках сайтов. Так появились клиентские сервисы аутентификации, которые работают в тот момент, когда пользователь нажимает на кнопку “авторизоваться через вконтакте”.

Вероятно, первым подобным стандартом был OpenID https://openid.net/what-is-openid/ . Этот стандарт предполагает, что каждый пользователь имеет уникальный FQDN, например, https://pavel.stepanoff.info который можно использовать для аутентификации через сайт, который его выдал (т.е. в данном случае через сайт http://stepanoff.info), который выступает в качестве центра управления безопасностью. Таким образом, не нужно вести имя пользователя и пароль на сайте, поддерживающем аутентификацию через OpenID, достаточно иметь на нем профиль и авторизоваться через сайт, выдавший этот OpenID.

Более современным решением являются OAuth и OAuth2– стандарты аутентификации, определенные в RFC-6749 https://tools.ietf.org/html/rfc6749 . Ключевая разница со стандартом OpenID заключается в том, что OpenID центр аутентификации подтверждает личность пользователя, в то время как для OAuth/OAuth2 центр авторизации является также поставщиком API, через которое при наличии подтвержденных полномочий можно получать некоторую пользовательскую информацию. По этой схеме, в частности, работает аутентификация через социальные сети – авторизовавшись, пользователь разрешает сторонним приложениям читать его профиль. Появившись как развитие OpenID для некоторых специальных случаев, стандарты быстро завоевали широкую популярность (в связи с развитием социальных сетей).

Проводя сравнение аутентификации через клиент и аутентификации через сервер, можно отметить, что первая является в первую очередь только средством аутентификации, но не авторизации. Используя директории пользователей, можно назначать полномочия пользователям, здесь же в качестве основной задачи ставится удостоверение факта, что пользователь – это тот, за кого он себя выдает; принципиально нет ограничений кто именно может создать свой профиль на сайте, используя OAuth2 и невозможно назначить пользователю полномочия раньше, чем этот профиль будет создан.

SSO аутентификация.

С развитием облачных технологий, особенно SaaS, возникла новая проблема – организации может быть необходимо иметь единый список пользователей, которым могут назначаться права в сторонних приложениях. Например, пользователь может работать с системами документооборота, управления предприятием и другими, предоставляемыми в рамках SaaS, однако было бы крайне неудобно управлять полномочиями в каждом приложении отдельно. Эта задача решается с помощью аутентификации SSO – Single Sign On. В этой модели аутентификации существует некоторый провайдер, удостоверяющий личность и полномочия пользователей, интегрированный с используемыми приложениями таким образом, что, удостоверив в нем свои полномочия, пользователь получает доступ ко всем приложениям сразу. Подобные системы (Identity Management as a Service) поставляют многие компании - интеграторы.

SSO аутентификация решает сразу большое количество проблем, такие как необходимость для пользователя помнить много разных паролей, создание сертифицированной системы безопасности (так как SSO позволяет закрыть сервера файрволлом, пропускающим только аутентифицированные запросы), упрощает поддержку и обслуживание систем авторизации и аутентификации.

Реализация аутентификации и авторизации средствами Spring.

Спринг реализует аутентификацию и авторизацию с помощью специального фильтра, называемого spring security. Этот фильтр внутри себя содержит свою собственную цепочку фильтров, проверяющую данные о пользователе, с которыми выполняется запрос. В случае, если они признаются недостоверными, фильтр отвергает запрос с соответствующей ошибкой (403 или 405 в зависимости от ситуации).

Для того, чтобы подключить стандартную аутентификацию и авторизацию к приложению spring-boot, на всем приложении используется аннотация **@EnableGlobalMethodSecurity(securedEnabled = true)** . После ее применения можно аннотировать отдельные методы указывая роли, которые должен иметь выполняющий их пользователь: **@Secured(<имя роли>).** Если пользователь не имеет этой роли, попытка вызова

метода приведет к **AccessDeniedException**. Другим вариантом проверки полномочий является указание в параметрах метода переменной типа Principal – эта проверка является более гибкой, так как позволяет в зависимости от полномочий возвращать разные данные.

Подробно данный вопрос рассмотрен по ссылке https://spring.io/guides/topicals/spring-security-architecture/

Рассмотрим реализацию технологии form-based authorization средствами spring+thymeleaf (библиотеки рендеринга страниц на основе шаблонов). В рамках решения задачи требуется решить несколько подзадач:

1. Настроить безопасность (b security)
2. Настроить HTTPS
3. Настроить защиту CSRF
4. Настроить форму логина
5. Защитить чувствительные ресурсы

Настройка безопасности приложения осуществляется с помощью специального конфигурационного бина:

Данный конфигурационный бин содержит несколько важных секций. Во-первых, метод authenticationManagerBean() определяет бин, который будет предоставлять сервис аутентификации. Как можно видеть, этот бин непосредственно связан с конфигурационным бином; смысл этого действия заключается в том, что настройки, определенные в конфигурационном бине, будут переданы в те места, где требуется аутентификация через автоматическое связывание.

Следующим важным методом является configure(AuthenticationManagerBuilder auth) . В этом методе можно определять способ проверки того, что пользователь принадлежит системе. В нашем случае используется “in-memory authentification”, то есть данные пользователя закодированы непосредственно в приложении. Этот способ аутентификации можно

использовать только при разработке ПО! В остальных случаях здесь должен находиться код, реализующий выбранную схему аутентификации, например, код получает пользователя из базы и проверяет его пароль. Связанный с этим метод configAuthentication(AuthenticationManagerBuilder auth) устанавливает алгоритм хеширования паролей BCrypt, и именно в этом формате пароль должен лежать в базе.

Наконец, еще один важный метод configure(HttpSecurity http) устанавливает свойства протокола, используя паттерн Builder. В нашем случае файл application.proiperties содержит следующие настройки:

```
server.port: 8443
server.ssl.key-store: target/keystore.p12
server.ssl.key-store-password: mypassword
server.ssl.keyStoreType: PKCS12
server.ssl.keyAlias: tomcat
```

что приводит к тому, что используется протокол https. Сертификат, ссылка на который идет из файла свойств, генерируется maven с помощью плагина, описанного ранее. Дальнейший код по настройке HttpSecurity указывает, какие именно страницы отвечают за логин, как называются переменные, представляющие логин и пароль, а также другие параметры. Обратите внимание, что по умолчанию CSRF включен (и это правильно). CSRF можно отключить с помощью команды .and().csrf().disable(), однако это сразу делает приложение незащищенным.

В новом Spring 3 данный конфигурационный бин выглядит похоже, но все же несколько отличается, например:

@Configuration

@EnableWebSecurity

public class SecurityConfiguration {

```
    @Bean
    public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
        http.authorizeHttpRequests(authorizationManagerRequestMatcherRegistry
                -> authorizationManagerRequestMatcherRegistry.requestMatchers("/js/**", "/css/**",
"/schools", "/public/rest/*").permitAll()
                        .anyRequest().authenticated())
                .formLogin(httpSecurityFormLoginConfigurer
                        -> httpSecurityFormLoginConfigurer
                        .loginPage("/login")
                        .usernameParameter("login")
                        .passwordParameter("pass")
                        .permitAll()
                        .successForwardUrl("/schools"))
                .logout(httpSecurityLogoutConfigurer -> httpSecurityLogoutConfigurer.permitAll()
                .logoutRequestMatcher(new AntPathRequestMatcher("/logout"))
                .logoutSuccessUrl("/login"));
        return http.build();
    }
    public void configureGlobal(AuthenticationManagerBuilder auth, PasswordEncoder
passwordEncoder) throws Exception {
        auth.inMemoryAuthentication()
```

```
        .withUser("user")

        .password(passwordEncoder.encode("password"))

        .roles("ALL")

        .and()

        .withUser("admin")

        .password(passwordEncoder.encode("admin"))

        .roles("ALL");

  }

}
```

Обработка страницы логина в thymeleaf будет выполнена автоматически, также автоматически будет поддерживаться токен CSRF. Шаблон должен лишь содержать поля ввода, имена которых совпадают с именами, использованными при настройке HttpSecurity, а также ссылку на форму логина:

```
<div class="main-div">
       <div class="panel">
              <h2 class="form-heading">Our University Database</h2>
              <p>Please enter your login and password</p>
              <div th:if="${param.error}"  class="alert alert-danger">Unknown
username or password.</div>
              <div  th:if="${param.logout}" class="alert alert-success">Good
bye!</div>
       </div>
       <form id="Login" th:action="@{/login}" method="post">
       <div class="form-group">
              <input type="text" class="form-control" id="login" name="login"
placeholder="Login"/>
       </div>
       <div class="form-group">
              <input type="password" class="form-control" id="pass"
name="pass" placeholder="Password"/>
       </div>
       <button type="submit" class="btn btn-primary">Login</button>
```

```
        </form>
</div>
```

Важным моментом является логаут. Для его выполнения необходимо выполнить http метод POST.

```
<form name="logoutForm" th:action="@{/logout}" method="post">
<input type="submit"  class="btn btn-primary" value="Sign Out"/>
</form>
```

В составе библиотеки Thymeleaf, рассматриваемой в следующей лабораторной работе, есть специальная разметка, которая позволяет понять - аутентифицирован пользователь или нет:

Код рендерится если пользователь не залогинен

Код рендерится если пользователь залогинен

Последнее, о чем следует сказать, это связь токена CSRF с AJAX-запросами. Для того, чтобы поддержать эту функциональность нужно проинформировать AngularJS о необходимости включения заголовок CSRF во все запросы к серверу, иначе они будут отвергнуты:

```
var app = angular.module('имямодуля', []).config(function ($httpProvider) {
        csrftoken = $("meta[name='_csrf']").attr("content");
        csrfheader = $("meta[name='_csrf_header']").attr("content");
        $httpProvider.defaults.headers.common["X-CSRF-TOKEN"] = csrftoken;
});
```

Задание на лабораторную работу.

1. Используйте в качестве основы лабораторную работу номер 3
2. Переведите вебсервер на использование SSL
3. Реализуйте form-based authentication
4. Защитите ресурсы, требующие запись и введите аудит.
5. Форма должна содержать кнопку Logout.

Лабораторная работа №5. Разработка приложения с использованием thymeleaf

Шаблоны страниц JSP.

Как мы уже выяснили в первой лабораторной работе, сервлеты могут генерировать HTML код путем отправки соответствующего текста в поток вывода, связанный с ответом сервера. Тем не менее, этот способ весьма неудобен для разработчика по нескольким причинам. Во-первых, необходимо контролировать корректность html документа, построенного путем конкатенации строк. Во-вторых, невозможно передать такой код графическим дизайнерам и наоборот – невозможно легко передать результат работы дизайнеров программистам. Также необходимо прямо в коде сервлета уметь поддерживать локализацию. Таким образом, использование сервлета для прямой генерации HTML документов является неудобным и ему на смену пришли генерации страниц на основе шаблонов, заполняемых данными с сервера.

Одной из первых подобных технологий явилась технология Java Server Pages или JSP. В этой технологии страница описывается специальным шаблоном, который является xhtml со встроенным кодом на языке Java. Такой код встраивается с помощью управляющих тегов вила <%@>, <@!> или <@=>. В результате шаблон страницы может выглядеть примерно так:

```
<%@ page language="java" contentType="text/html; charset=UTF-8"
pageEncoding="UTF-8"%>
<%@ page import="java.util.*, java.text.*" %>
<%!
String getNow() {
    return new SimpleDateFormat("dd.MM.yyyy hh:mm:ss").format(new Date());
} %>
<html>
  <head>
      <meta http-equiv="Content-Type" content="text/html; charset=UTF-8">
  </head>
  <body>
       <h1> Сейчас <%= getNow(); %></h1>
  </body>
</html>
```

Такое встраивание Java кода непосредственно в страницу становится возможным потому, что на самом деле специальный транслятор генерирует по этому шаблону сервлет и встраивает в него соответствующий код. Таким образом, код на языке Java, вставленный в разметку,

естественным образом становится частью сервлета. При этом на этапе создания шаблона достаточно сложно контролировать синтаксическую правильность этого кода, то есть полученный сервлет может и не скомпилироваться.

Spring MVC.

MVC (Model-View-Controller, Модель-представление-контроллер)– популярный паттерн при разработке интерфейсов пользователя. Его основная идея заключается в изоляции друг от друга модели, представляющей данные системы, представления, то есть графического интерфейса и контроллера, реализующего бизнес-логику. За счет такого подхода существенно упрощается внесение изменений в приложение, так как изменение одной из частей не обязательно тянет за собой изменений двух остальных. Например, изменив представление, можно не менять бизнес-логику и модель. Дополнительным преимуществом подхода MVC является возможность простой реализации нескольких представлений у одной и той же модели.

Как мы можем видеть выше, JSP очень слабо соответствует шаблону MVC – как минимум, отделить бизнес-логику от представления крайне тяжело. Фреймворк Spring MVC является механизмом, позволяющим абстрагировать механизмы генерации страниц от моделей и контроллеров. В этом подходе контроллер является кодом, подготавливающим данные для формы, реализованным в виде бина с аннотацией @Controller. Контроллер возвращает данные в виде объекта класса ModelAndView, содержащего всю информацию, необходимую для генерации экранной формы по шаблону. Собственно форма может генерироваться с помощью одной из целого ряда технологий – в частности, JSP, Thymeleaf, Groovy, FreeMarker, Jade. Конкретная технология определяется тем, какие именно в приложении реализованы бины конфигурации. Мы будем рассматривать в дальнейшем генерацию страниц средствами Thymeleaf.

Шаблоны страниц Thymeleaf.

Для подключения Thymeleaf необходимо подключить соответствующий артефакт

```
<dependency>
```

```
    <groupId>org.springframework.boot</groupId>

    <artifactId>spring-boot-starter-thymeleaf</artifactId>

</dependency>
```

Этот артефакт определяет реализацию двух бинов, необходимых для функционирования Thymeleaf – SpringTemplateEngine и SpringResourceTemplateResolver. По умолчанию, все шаблоны thymeleaf должны находиться в каталоге /resources/templates , а все статические ресурсы (неизменяемые скрипты, стили, картинки) в каталоге /resources/static . Обратите внимание, что использование thymeleaf не исключает одновременное использование и других систем шаблонизации страниц.

Thymeleaf оперирует понятием “естественного шаблона”. Это означает, что шаблон Thymeleaf является одновременно допустимым ресурсом того вида, который он определяет. Например, шаблон страницы является допустимой страницей, шаблон скрипта является скриптом и так далее. Это достигается за счет использования в качестве управляющей разметки последовательностей символов, которые игнорируются соответствующими средами исполнения. В частности, для управления содержимым страницы используются атрибуты, принадлежащие пространству имен “th:” из xmlns:th="http://www.thymeleaf.org".

Самый просто способ использования thymeleaf следующий. Если в html теге шаблона имеется некоторый атрибут, скажем, value, то соответствующий ему атрибут th:value позволяет при генерации страницы по шаблону получить значение этого атрибута с сервера. Например, это может выглядеть так:

```
<input name=”quantity” value=”0” th:value=”код thymeleaf по генерации значения”/>
```

Обратите внимание, что шаблон должен быть допустимым xhtml файлом, поэтому даже те теги, которые обычно не имеют закрывающих (<input>,
 и тому подобные) все равно должны иметь закрывающий тег. Такой подход с заменой атрибутов позволяет работать над

шаблоном графическим дизайнерам, так как они могут сразу видеть то, как будет выглядеть сгенерированная страница без компиляции приложения.

Отдельно следует упомянуть некоторые специальные атрибуты thymeleaf. Например, атрибут th:text позволяет управлять при генерации страницы содержимым текста, расположенного внутри тега. Например, это может выглядеть так: <p th:text=”новый текст”>старый текст . Еще один полезный атрибут th:if управляет тем, помещается ли тег в генерируемый документ или нет, в зависимости от значения, пришедшего с сервера. Еще два важных атрибута – th:object и th:inline будут рассмотрены позже.

Следующая важная задача, которая возникает при использовании Thymeleaf – это собственно получение данных с сервера. Для того, чтобы подставить вместо атрибута некоторые данные, необходимо указать, откуда их брать. Эта задача решается с использованием специального языка Thymeleaf. В самой простейшей форме мы будем использовать ссылки вида $, # и @.

Ссылка вида ${имя_параметра} определяет, что значением соответствующего атрибута thymeleaf является значение свойства “имя_параметра”, которое контроллер формы указал в возвращенном ModelAndView. Например, мы можем написать в шаблоне страницы tmpl.html

```
<div  th:text="${param}">Здесь будет какой-то текст</div>
```

И определить контроллер формы следующим образом:

```
  public ModelAndView tmplController() {
     ModelAndView mav = new ModelAndView("tmpl");
     mav.addObject("param", “новый текст”);
     return mav;
  }
```

В результате значение “новый текст” заменит текущее значение текста в теге .

Ссылка вида #{имя_параметра} является ссылкой на файл локализации.

Задание на лабораторную работу.

1. Реализуйте задачи из лабораторной работы 4 средствами Thymeleaf
2. Обратите внимание на локализацию приложения. Русский язык должен поддерживаться. Локализация должна выполняться средствами Thymeleaf.

Pavel A. Stepanov, pavel@stepanoff.info © 2018 -2023
Кафедра компьютерных технологий и программной инженерии (№43) СПб ГУАП

Лабораторная работа №6. Разработка приложения с использованием Hibernate

Основные понятия JDBC. Соединение с СУБД.

JDBC является технологией, лежащей в основе более высокоуровневых средств доступа к базам данных, таких как ORM. Основная идея JDBC состоит в том, что все конкретные детали работы с базой данных абстрагирует предоставляемый ее производителем драйвер, а программист работает через набор стандартных классов, образующих обертку вокруг него. За счет этого обеспечивается некоторая унификация доступа к БД, хотя и в меньшей степени, нежели для ORM (например, все равно надо иметь в виду особенности версии SQL).

В основе работы с JDBC лежит класс DriverManager. Этот класс абстрагирует всю работу с драйверами баз данных и позволяет создавать соединения с СУБД, используя перечни свойств или URL баз данных. Такое соединение представлено классом java.jdbc.Connection. Для инициализации этого класса требуется информация, специфичная для производителя БД. Обычно при работе из среды аппсервера созданием экземпляров этого класса управляет именно контейнер, а не пользовательский код, соответственно, приложение тем или иным образом его от контейнера получает. Это связано с тем, что получение соединения с СУБД является тяжелой операцией, а сами соединения требуют ресурсов, поэтому необходимо ими эффективно управлять и переиспользовать. Эту задачу решают пулы соединений. Одним из наиболее популярных решений для переиспользования соединений с БД является Hikari pool.

Следующим важным классом для работы с данными является класс, представляющий запрос. Таких классов три вида – Statement, PreparedStatement и CallableStatement, последний из который предназначен для исполнения хранимых процедур. Statement и PreparedStatement представляют обычный SQL запрос. Ключевое отличие этих двух классов заключается в том, что PreparedStatement позволяет безопасно параметризовать запрос (класс Statement уязвим к SQL инъекциям), выполнить запрос несколько раз с разными параметрами, построив его план только единожды, а также в случае непараметризованных запросов выполнять их

потокобезопасно в параллель. Таким образом, во всех случаях необходимо использовать PreparedStatement, а не Statement.

Примерная схема работы с PreparedStatement выглядит следующим образом:

```
//Шаг 1. Создание оператора из соединения

PreparedStatement myStatement

 = myConnection.prepareStatement(“select * from students where id = ?”);

//Шаг 2. Параметризация оператора

myStatement.setInt(1,10);

//Шаг 3. Выполнение и получение результата

ResultSet myResultSet= myStmt.executeQuery();
```

Возвращенный объект ResultSet связан с курсором в базе данных и реализует паттерн итератор. С его помощью можно получить все данные, возвращенные запросом. Например, код может выглядеть так (предполагая, что таблица students из двух полей – id и name):

```
while (myResultSet.next()) {
        Integer id = myResultSet.getInt(1);
        String name = myResultSet.getString(2);
}
```

Многие начинающие программисты испытывают желание сформировать запрос путем конкатенации строк и параметров, то есть, например, в форме

```
myStatement = myConnection.prepareStatement(“select * from students where id = “+x);
```

где x – значение ключа поиска. Такая форма запроса является одним из наиболее часто встречающихся антипаттернов, так как не только лишает разработчика возможности повторно выполнить запрос с другим параметром, но и потенциально уязвима к инъекции SQL.

Еще одним важным действием, о котором необходимо помнить при работе с СУБД, является закрытие ресурсов. Соединения с СУБД, PreparedStatement и ResultSet потребляют ресурсы СУБД (особенно последний, так как он может быть связан с открытым курсором в базе). Поэтому по завершении работы с ними их обязательно необходимо закрыть, выполнив метод close().

JDBC Template.

Как можно видеть из вышеприведенного описания, прямое использование JDBC очень многословно и сопряжено с написанием значительного количества сервисного кода. Поэтому был предложен подход, позволяющий представить часть работы с базой данных через стандартные механизмы Spring, получивший название JDBC Template.

Первым важным отличием подхода является представление источника данных через бин, например, так:

```
@Bean
public DataSource dataSource() {
return new EmbeddedDatabaseBuilder()
.setType(EmbeddedDatabaseType.H2)
.addScript("classpath:jdbc/create-schema.sql")
.addScript("classpath:jdbc/data.sql").build();
}
```

Теперь ряд действий существенно упрощается. Например, для добавления записи можно использовать метод, аналогичный следующему:

jdbcTemplate.update("INSERT INTO STUDENT VALUES (?, ?)", id, name); }

Выборка в JDBC Template позволяет преобразовывать возвращенный курсор непосредственно в JAVA – объект, содержащий его данные. В случае, если запрос возвращает простой тип, например, число, его результат может быть преобразован автоматически:

```
int result = jdbcTemplate.queryForObject( "SELECT COUNT(*) FROM STUDENTS", Integer.class);
```

Для более сложных преобразований используется специальный класс – маппер, предоставляющий отображение между полями ResultSet и полями класса, например:

```
public class StudentRowMapper implements RowMapper<Employee> {
        @Override
        public Student mapRow(ResultSet rs, int rowNum) throws SQLException {
                Student student = new Employee();
        student.setId(rs.getInt("ID"));
        student.setName(rs.getString("NAME"));
        return employee;
        }
}
```

Далее этот класс может быть использован следующим образом:

```
String query = "SELECT * FROM STUDENT WHERE ID = ?";
Student student = jdbcTemplate.queryForObject( query, new Object[] { id }, new StudentRowMapper());
```

ORM. Виды ORM. Стандарт JPA.

Глядя на вышеприведенный код маппера, можно легко прийти к выводу, что он очень многословен и неудобен. Было бы гораздо удобнее писать код, если бы существовала возможность автоматического преобразования полей базы данных в атрибуты класса. Эту идею взяли на вооружение разработчики другой технологии – ORM или Object Relation Mapping, в которой весь обмен с базой данных идет через объекты. Для языка Java определен стандарт такого взаимодействия, который называется JPA (Java Persistence API).

Существуют десятки библиотек ORM для Java. Наиболее известной и популярной ORM является Hibernate, поддерживающая стандарт JPA. Достаточно часто встречаются MyBatis и EclipseLink, последняя считается эталонной реализацией JPA.

Рассмотрим архитектуру доступа к данным JPA. В основе доступа к данным находится класс, который называется Persistence, который содержит в себе информацию о конфигурации соединения с базой данных. Этот класс нужен для конструирования экземпляра класса EntityManagerFactory, в свою очередь, являющегося фабрикой экземпляров классов EntityManager. Этот последний класс является фасадом устойчивого контекста (persistent context) базы данных, то есть именно через него и осуществляется все взаимодействие приложения с базой. В частности, именно он реализует четыре основных операции с базой данных, получившие аббревиатуру CRUD (Create, Read, Update, Delete) с помощью методов persist (создать или обновить), remove (удалить) и find (найти).

Единицей взаимодействия с базой данных является объект-сущность, являющийся представлением таблицы. Атрибуты класса такого объекта размечены с помощью специальных аннотаций, указывающих на правила связывания их с полями базы данных. Среди таких аннотаций следует отметить @Id, описывающую ключевые поля, @Column, описывающую данные поля базы данных, @OneToMany и @ManyToOne, описывающию внешние ключи,что позволяет прозрачно для пользователя выполнять запросы к связанным таблицам, просто проходя по ссылкам внутри таких объектов. Кроме того, на класс такого

объекта налагаются некоторые ограничения, например, наличие конструктора без аргументов, геттеров для всех полей и поля с аннотацией @Id (что особенно удобно при использовании проекта Lombok и аннотации @Data либо подобных). Такой объект может существовать в двух основных состояниях – присоединенном и отсоединенном; если объект присоединен к базе данных, то операции над ним могут отражаться на этой базе данных, например, при запросе значения внешнего ключа может автоматически выполняться запрос к базе. Пример такого класса, представляющего сущность, приведен ниже:

```
@Entity
@Table(name = "School")
@Data

public class School implements Serializable {
   private static final long serialVersionUID = 1L;
   @Id
   @Column(name = "school_id")
   @GeneratedValue(strategy = GenerationType.IDENTITY)
   private Integer id;

   @OneToMany(targetEntity = Department.class, mappedBy = "school" , fetch = FetchType.LAZY)
   private List<Department> deptsList;

   @Column(name = "school_number", unique=true)
   private Integer number;

   @Column(name = "school_name")
   private String name;
}
```

ORM, при всем своем удобстве, не свободна от проблем. В частности, достаточно сложно реализовать эффективную работу с запросами к СУБД, оперируя только сущностями - таблицами. Ответом на эту проблему являются JPA запросы, позволяющие преобразовать результат запроса к базе данных в объект специального типа. Другой известной проблемой является так называемая проблема n+1. Если объект содержится в виде ссылки в n записей в связанной таблице, то получение его из базы данных приведет к выполнению n+1 запроса (для самого объекта и каждой связанной записи). Частичным ответом на эту проблему является использование свойства внешнего ключа FetchType.LAZY, что позволяет получать связанные записи из базы только в том случае, если они потребовались в коде. Тем не менее, ручная реализация такого кода в виде двух запросов, сопоставляемых вручную, на порядок эффективнее, нежели стандартные средства JPA.

Еще одним важным следствием появления ORM является паттерн DTO (Data Transfer Object). В ряде случаев необходимо просто прочитать данные из базы и вернуть через REST на клиент. Однако, вернуть напрямую возвращенную базой запись из таблицы невозможно, так как при попытке трансформации в JSON будут прочитаны все внешние ключи и потенциально возвращенная информация будет гораздо больше по объему, чем необходимо, либо вообще процесс зациклится, если в базе есть циклические зависимости. Для решения этой проблемы используется большое количества разнообразных техник – отключение записи от базы перед трансформацией, специальные аннотации, информирующие JSON трансформер о том, что не нужно идти по внешним ключам, но наиболее универсальным является DTO, то есть преобразование объекта в другой объект (который и называется DTO), содержащий только нужную информацию и не имеющий связи с базой данных. Это преобразование обычно осуществляется специальным маппером, инкапсулирующим бизнес-логику того, что именно нужно возвратить клиенту.

Поддержка ORM в Spring.

В виду того, что ORM позволяет в простых случаях существенно ускорить разработку интеграций с СУБД, эта технология имеет широкую поддержку на платформе Spring. Основным механизмом поддержки является библиотека Spring Data JPA.

Подключение библиотеки (для spring-boot) может быть выполнено следующим артефактом:

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-data-jpa</artifactId>
</dependency>
```

После этого пользователю становятся доступны следующие важные возможности.

Во-первых, контекст базы данных может быть сконфигурирован через файл application.properties , например, следующим образом:

spring.datasource.url=jdbc:h2:mem:testdb

spring.datasource.driverClassName=org.h2.Driver

spring.datasource.username=sa

spring.datasource.password=password

spring.jpa.database-platform=org.hibernate.dialect.H2Dialect

В случае если, к примеру, требуется сконфигурировать два контекста базы данных, это тоже возможно с использованием конфигурационных бинов, которые укажут разные иерархии классов для разных настроек. Подробнее об этом можно прочесть в документации.

Следующая полезная функция Spring Data JPA – поддержка так называемых JPA репозиториев, избавляющих пользователя от непосредственной работы с EntityManager. Эта поддержка включается аннотацией @EnableJPARepositories и позволяет создавать интерфейсы, аннотированные с помощью @Repository, которые автоматически генерируют запросы к базам данных на основании аннотаций и соглашений об именах. Например, рассмотрим слеюущий интерфейс:

```
public interface SchoolRepository extends CrudRepository<School, Integer> {

   School findByNumber(Integer number);

}
```

При попытке автосвязывания этого интерфейса спринг сгенерирует прокси объект, обладающий следующими возможностями.

Во-первых, он будет работать с таблицей, которая через аннотацию @Table связана с классом School, при этом ключ будет иметь тип Integer.

Во-вторых, будут создан ряд CRUD запросов, которые содержатся в качестве стандартных в интерфейсе CrudRepository, в частности, save, deleteById, findById и ряд других.

Наконец, метод findByNumber будет автоматически преобразован в поисковый SQL запрос в соответствии с правилами именования. Полный набор правил наименования можно получить в документации https://docs.spring.io/spring-data/jpa/docs/current/reference/html/#jpa.query-methods . К примеру, метод findDistinctByFirstnameOrLastname будет соответствовать примерно следующему SQL: select distinct * from Student where Firstname = ?1 or Lastname = ?2

Поддержка транзакций

Важным вопросом при работе с базой данных является поддержка транзакций. В Spring Data JPA она реализуется с помощью аннотации @Transactional, которая, будучи поставлена на метод, управляет контекстом транзакции внутри этого метода и всех вызываемых им методов, при условии, что эти вызовы осуществлены через стандартные механизмы связывания Spring (то есть не используется прямое конструирование объекта методом new). Для аннотации @Transactional важными свойствами являются признак только чтения, распространение и уровень изоляции.

Транзакция может быть помечена как только для чтения с помощью свойства readonly, а именно, **@Transactional**(readonly=true). Такая транзакция более эффективна на стороне СУБД,

так как не требует от базы реализации по умолчанию некоторых механизмов разграничения доступа. При этом, обычно, ничто не запрещает в рамках такой транзакции модифицировать базу – большинство СУБД может перестроится налету.

Распространение транзакции – это алгоритм, которым Spring руководствуется при вызове метода с аннотацией @Transactional из другого метода с такой же аннотацией. Различают следующие основные типы распространения:

- Propagation.REQUIRED – поведение по умолчанию, если транзакции не было, то она создается, если была, то используется существующая
- Propagation.REQUIRES_NEW – всегда создается новая транзакция
- Propagation.MANDATORY – если текущей транзакции нет, выбрасывается исключение
- Propagation.NEVER – если есть текущая транзакция, выбрасывается исключение
- Propagation.SUPPORTS – если транзакция есть, она используется, иначе режим auto commit
- Propagation.NOT_SUPPORTED – исполняется вне контекста транзакции, если она существует, в режиме auto commit
- Propagation.NESTED – создается вложенная транзакция, поведение в основном похоже на REQUIRES.

Наконец, свойство isolation позволяет указать один из четырех уровней изоляции, используемых СУБД

- Isolation.DEFAULT – уровень изоляции, которую раза данных поддерживает по умолчанию, например, в Oracle и Postgres это read commited, в MySQL это repeatable read;
- Isolation.READ_UNCOMMITED
- Isolation.READ_COMMITED
- Isolation.REPEATABLE_READ

- Isolation.SERIALIZABLE

Также важно иметь в виду, что Spring автоматически управляет подтверждением и откатом транзакции. Если в контексте вызова метода с аннотацией @Transactional произойдет RuntimeException или его потомок, то транзакция автоматически откатится.

Паттерн DTO.

Начинающий программист испытывает соблазн использовать в API своего сервиса те же самые классы, что и для доступа к базе данных. В общем случае это очень плохое решение, так как внешние ключи сущностей могут так же быть возвращены на клиент, а это значит, что вместо объекта вернется дерево. Более того, если между сущностями имеются циклические зависимости по ключам, то это может привести к попытке вернуть результат бесконечного размера и последующим падением с исчерпанием памяти.

Для решения указанной проблемы используется шаблон Data Transfer Object или просто DTO. Шаблон состоит в том, что для API используется специализированный класс, который не связан с базой данных и не решает больше никаких задач. Необходимые значения при необходимости копируются между объектами этого класса и объектами-сущностями. Более того, если имена полей в этом классе имеют те же имена, что и в сущности, то можно использовать специальный сервисный класс ObjectMapper, который скопирует поля с одинаковыми именами, например, так:

```
ObjectMapper objectMapper = new ObjectMapper();

ObjectA objA = objectMapper.convertValue(objB, ObjectB.class);
```

В этом примере все одинаковые поля из объекта objB класса ObjectB будут скопированы в объект objA класса ObjectA. При этом ObjectB должен иметь на них геттеры, а ObjectA – сеттеры.

Задание на лабораторную работу.

1. Используйте в качестве основы лабораторную работу 4 или 5 (для заочников – 3).
2. Добавьте базу данных, модель данных, JPA репозитории и сервисы. Должно быть не менее двух таблиц и одного внешнего ключа.
3. Реализуйте операции чтения/записи с помощью JPA (spring-boot-data-jpa, также допустимо spring- boot-data-jdbc если сумеете объяснить разницу)

Лабораторная работа №7. Разработка приложения с асинхронной очередью сообщений

Архитектура, управляемая событиями (EDA, Event Driven Architecture).

При взаимодействии через REST достаточно серьезной проблемой является синхронность взаимодействия, вызывающая пропорциональный рост нагрузки на сервер с увеличением числа запросов. Если клиентская система начинает генерировать нагрузку, которую нижележащий сервер не в состоянии обработать, такая нагрузка может снижаться средствами ограничения трафика. Однако есть и неустранимые проблемы, например, если в рамках обработки какого-либо запроса должны быть задействованы несколько систем, то при синхронном взаимодействии время обработки одного запроса будет складываться из времен обработки всех запросов, участвующих во взаимодействии.

Решением проблемы может является перевод части действий, не формирующих непосредственный ответ API, в асинхронный режим. Типовыми случаями являются задачи нотификации таких частей системы, как журналирование, биллинг, BI о произошедших при обработке существенных изменениях, которые могут осуществляться путем посылки сообщения, которое обрабатывается асинхронно вне контекста запроса.

В ряде случаев, для систем, которым требуется особенная устойчивость к нагрузкам, синхронное взаимодействие может полностью меняться на асинхронное (и, соответственно, взаимодействие с ними клиентов будет осуществляться не по протоколу http, а через вебсокеты). В этом случае клиент имеет некоторую модель состояния сервера, а сервер его время от времени обновляет. Например, пользователь работает с чатом и нажимает на кнопку отправить. Клиент отправляет событие на бэкенд и показывает отправленное сообщение в чате, однако это вовсе не означает, что это сообщение действительно дошло до какого-либо из собеседников или даже вообще до чата. Событие будет обработано асинхронно и сервер в определенный момент пошлет обратно событие о том, что предыдущее событие было обработано, чтобы клиент мог это отобразить его состояние пользователю. Соответственно, для остальных пользователей чата на их клиенты придет событие, информирующее их о сообщении первого пользователя. В такой модели важной проблемой является потеря

событий, в результате чего промежуточные состояния могут быть потеряны. Для борьбы с этой проблемой все события имеют сквозную нумерацию, и если клиент получает событие с номером, которое превышает номер последнего полученного события более, чем на единицу, он перезапрашивает полное состояние через REST.

Апофеозом этого подхода является архитектура Event Sourcing, которая часто применяется для сильно нагруженных систем. В этой архитектуре приложение не имеет материализованного состояния (за исключением, быть может, моментов начала сессии работы и окончания сессии работы), а при необходимости его получения это состояние собирается из событий, которые это состояние меняют. Представим работу онлайнового редактора документа, такого как Google Docs. Множество пользователей одновременно редактирует один документ. Для того, чтобы получить финальную версию документа все действия всех пользователей могут быть представлены в виде микроизменений, которые выстроены в цепочку и упорядочены. Теперь каждый клиент может получить текущее представление документа, в том числе и в режиме псевдореального времени, путем повторения событий изменения, приходящих к нему непрерывным потоком. В ситуации, когда некоторый объект подвергается постоянным изменениям, архитектура Event Sourcing обеспечивает серьезное преимущество в производительности по сравнению с материализацией состояния.

Брокеры сообщений. Основные модели обмена сообщениями.

Брокеры сообщений – широкий класс программных средств, предназначенных для поддержки передачи сообщений на бэкенде (а иногда – также и на фронтенде). На основе брокеров сообщений строятся прикладные программные системы, основанные на событиях.

Основой работы с брокером сообщений является его контракт. В зависимости от возможностей и настроек брокера он может поддерживать или не поддерживать надежную доставку, сохранение порядка сообщений, пакетную обработку, доставку событий на веб-клиент, перечитывание очереди сообщений. Каждый брокер имеет свою собственную модель

работы с сообщениями, в результате чего для выбора подходящего брокера требуется хорошее понимание принципов его работы.

При обмене сообщениями используются две основные стратегии – писатели-читатели и производители-потребители. Модель читатели-писатели предполагает, что отправленное в канал сообщение будет доставлено всем подписанным на канал клиентам (писатель выпускает книгу и все его читатели ее читают). Модель производители-потребители предполагает, что отправленное в канал сообщение будет доставлено только одному потребителю, выбранному по определенным правилам (товар, произведенный производителем, покупается одним потребителем). Брокеры сообщений поддерживают эти две модели разнообразными способами.

Существует огромное количество брокеров сообщений, но наиболее встречающихся два – это RabbitMQ и Kafka. Кроме них можно отметить достаточно часто встречающиеся ActiveMQ и IBM MQ. Также важное значение имеет стандарт JMS (Java Message Service), который является одной из первых моделей обмена сообщениями и потому используется в большом количестве старых систем.

RabbitMQ.

RabbitMQ является системой, оригинально разработанной для работы по протоколу AMQP. Это бинарный протокол, в котором используется клиент-серверная коммуникация между каналом и клиентом канала в форме определенных команд, что сближает его с протоколами типа RPC. Сторона коммуникации, которая имеет сообщение для отправки, посылает команду стороне, которая должна его принять, и эта команда синхронно обрабатывается. Например, если сервис А хочет послать сообщение сервису Б, то он инициирует коммуникацию с RabbitMQ, после чего RabbitMQ в том же порядке инициирует коммуникацию с сервисом Б. За счет того, что все участники коммуникации не имеют представления о загрузке своих контрагентов, такая схема более уязвима к высоким нагрузкам; кроме того, сервер RabbitMQ должен инициировать все виды коммуникаций с клиентами канала, что приводит к тому, что пропускная способность всей системы ограничена его ресурсами CPU.

RabbitMQ оперирует понятиями точек обмена (Exchange) и очередей (Queue). Сообщение посылается в точку обмена, после чего по определенным правилам фильтрации передается в очередь, на которой слушает получатель этого сообщения. Очередь связывается с получателем и инициирует получение им сообщение. Если на очереди слушает несколько получателей и если для очереди сконфигурирована надежная доставка, то может получиться так, что сообщения перемешаются и более старые будут доставлены в одного получателя позже, чем более новые. Основной моделью RabbitMQ является модель производители-потребители, для реализации модели писатели-читатели требуется создание нескольких очередей, на каждой из которых будет слушать свой читатель. Также для точки обмена может быть настроена устойчивость, которая бывает двух видов – с гарантией доставки и с гарантией обработки.

В целом, RabbitMQ показывает себя достаточно сложным в настройке и слабо толерантным к высоким загрузкам, поэтому основным применением технологии являются не слишком загруженные системы, в которых требуется выполнение вычислительно тяжелых или длительных задач. Например, серверная генерация документов, изображений, сложные вычисления, работа с SOA могут выполняться с помощью RabbitMQ, так как основная нагрузка здесь ложится не на канал данных, а на потребителей сообщений. В задачах, в которых необходимо доставлять огромное количество сообщений, не требующих сложной обработки, например, чтобы показывать онлайн статус пользователей мессенжера, RabbitMQ существенно проигрывает Kafka.

Kafka.

Kafka является системой, устроенной по логике, во многом являющейся диаметрально противоположной RabbitMQ. Основной задачей Kafka является репликация журнала сообщений. Журнал сообщений публикуется в топике (topic) и нарезается на партиции (partition) в соответствии с настройками этого топика. Например, топик может быть сконфигурирован в четыре партиции, соответственно, журнал сообщений будет нарезан примерно на четыре части в соответствии с некоторой хеш-функцией. Партиция является

единицей масштабирования и устойчивости, то есть при необходимости обрабатывать большее количество данных необходимо добавить больше партиций.

Kafka гарантирует, что в рамках одной партиции сообщения не могут перемешаться, что достигается тем, что у одной партиции в рамках одной группы серверов может быть только один потребитель (группа серверов – это некоторый идентификатор). Если требуется реализовать модель писатели- читатели, то каждого читателя должна быть своя группа серверов.

Каждая партиция, в зависимости от сконфигурированной степени устойчивости топика, может обрабатываться одним или несколькими брокерами, входящими в кластер Kafka. Один из этих брокеров является мастером партиции, а остальные – репликами. Вопрос о том, какой именно брокер является мастером партиции, решает брокер – лидер топологии. Этот брокер до версии Kafka 3.0 выбирается с помощью кластера zookeeper, который является необходимой частью инсталляции. В версии 3.0 и выше Kafka имеет возможность самостоятельно решать проблему выбора лидера.

Клиенты (консьюмеры) топика распределяются по партициям; если их больше, чем партиций, часть клиентов простаивает. Если их меньше, чем партиций, то часть клиентов имеет несколько связанных партиций. Если какой-либо из клиентов отказал, выполняется перераспределение клиентов по партициям. Далее важным отличием от RabbitMQ является то, что клиент инициирует получение сообщений с сервера, а не наоборот. На стороне брокера содержится специальный системный топик, указывающий, до какого места была прочитана партиция определенной сервер группой, и начиная с этого места клиенту выдаются дальнейшие сообщения. Эта модель гарантирует, что события всегда будут обработаны в одном порядке в рамках партиции, а также дает возможность выполнять пакетную обработку и гарантированную обработку, так как клиент, обработав пакет сообщений, может сообщать мастеру партиции об этом факте (выполнять commit партиции).

Kafka не гарантирует высоких скоростей доставки индивидуальных сообщений, однако в случае пересылки большого объема информации ее пропускная способность, фактически,

ничем не ограничена. Кроме того, Kafka позволяет перечитывать топики, что делает ее сходной с базой данных, работать с парами ключ-значение (в рамках структуры данных LSM-дерево) и содержит такую важную технологию, как Kafka Connect – с помощью специальных библиотек Kafka может читать все изменения в базе данных и преобразовывать их в поток событий в Kafka для последущей обработки.

Задание на лабораторную работу.

Целью работы является реализация простой системы распределенной репликации (“писатели-читатели”).

1. Скачайте и разверните Apache Kafka
2. Модифицируйте свое приложение со встраиваемой базой данных так, чтобы его можно было запустить в нескольких экземплярах на разных портах
3. Реализуйте в рамках своего приложения Producer и Consumer такие, что
    a. Producer при каждой операции записи оповещает соответствующий топик
    b. Consumer при получении информации из топика записывает обновление в локальную (встроенную в приложение) базу
4. Продемонстрируйте, что информация, записанная одним приложением, доступна второму приложению.

Или альтернативное задание – настройка синхронизации средствами CDC –Change Data Capture (для владельцев мощных ноутбуков)

1. Скачайте и разверните Apache Kafka
2. Модифицируйте свое приложение так, чтобы оно работало с базой данных Postgres либо другой, поддерживаемой Kafka Connect. Сделайте так, чтобы на Вашем компьютере можно было запустить две базы данных и два разных экземпляра приложения (на разных портах).

3 Используя, например, Debezium, настройте канал из обеих баз данных в Kafka. Реализуйте консьюмеры так, чтобы изменения в каждой из баз отображались в обеих базах. Избегайте зацикливания обновлений.

4 Продемонстрируйте, что информация, записанная одним приложением, доступна второму приложению и наоборот.

Лабораторная работа №8. Разработка микросервиса

Виртуализация и связь с автоматическим развертыванием.

С развитием микросервисной архитектуры приложения получили сложную структуру развертывания, предполагающую установление связей между различными микросервисами и, потенциально, автоматическое масштабирование. Это, в свою очередь, привело к крайнему неудобству развертывания приложений на физических машинах. Можно отметить следующие причины, почему физические машины не подходят для микросервисной архитектуры:

1. Серверное оборудование крайне дорогое; писать приложения так, чтобы максимально эффективно его использовать достаточно сложно. Необходимые ресурсы для приложения могут быть существенно меньше, чем ресурсы одного сервера.
2. При необходимости наращивания мощностей приложения требуется физическая покупка сервера, физический монтаж сервера и физический монтаж сетевой инфраструктуры. Эти действия не подлежат автоматизации, при этом цикл покупки оборудования может занимать месяцы.
3. Крайне сложными и дорогими становятся схемы обеспечения высокой доступности, особенно связанные с распределением трафика по нескольким датацентрам.
4. Все вышесказанное осложняется ограниченными ресурсами датацентров, которые, обычно, имеют предел в наращивании веса оборудования, потребления питания и выделения тепла.

Все вышесказанное привело к идее виртуализации оборудования, то есть реализации виртуальных агрегатов развертывания, представляющих только части ресурсов вычислительной системы.

Первыми системами виртуализации были виртуальные машины, такие, как решения VMWare. Создавалась виртуальная машина, на которую устанавливалась собственная операционная система и которая не подозревала о том, что на самом деле она работает не на физическом

оборудовании. Такая виртуальная машина исполняла операционную систему, которая называлась гостевой, в отличие от хост-системы, запущенной на физической машине и исполнявшей гипервизор – собственно, приложение, обеспечивающее виртуализацию. Эта концепция была огромным прорывом вперед – на одном сервере можно было запустить десятки или даже сотни виртуальных машин, эффективно используя ресурсы оборудования; при необходимости через средства менеджмента систем виртуализации можно было увеличивать или уменьшать количество виртуальных машин, исполняющих те или иные микросервисы.

Контейнеризация. Docker.

Несмотря на преимущества виртуальных машин они, все-таки, не свободны от недостатков. Прежде всего, операционная система внутри виртуальной машины потребляет существенные ресурсы на работу тех сервисов, которые для работы приложения не нужны. Поэтому следующим шагом виртуализации явилась контейнеризация – выделение изолированных процессов в рамках хост-системы. Такая модель изоляции процессов иногда называется софтверной виртуализацией.

Процессы-контейнеры являются особыми процессами, которые максимально изолированы от адресного пространства процессов хост-системы. Они не являются виртуализацией оборудования в прямом смысле, вместо этого операционная система на логическом уровне контролирует их доступ к адресному пространству хост машины, создавая видимость того, что такой процесс работает один, в своей собственной файловой системе и своем собственном сетевом окружении. В виду особенностей реализации и архитектуры операционных систем наиболее эффективны такие процессы на ядре Linux.

На данный момент ведущей системой контейнеризации является Docker. Он состоит из трех основных компонент – клиента, хоста и реестра. Хост оперирует объектами контейнеризации, такими как образами (имиджами) контейнеров и их запущенными экземплярами. При необходимости хост может взять имидж контейнера и запустить, либо остановить уже

запущенный. Реестр – место, где хранятся образы. Клиент – приложение, позволяющее управлять работой Docker.

Выделяют следующие основные объекты, которыми манипулирует Docker:

Образ контейнера – сохраненный на диске или в реестре файл, содержащий информацию о том, как следует запускать построенный на его основе контейнер. В определенном смысле может рассматриваться как шаблон контейнера.

Контейнер – запущенный экземпляр образа.

Устойчивые тома – по умолчанию все контейнеры не имеют устойчивого состояния, то есть при рестарте любое их состояние пропадает. Решением этой проблемы является либо хранение состояние во внешней базе данных, либо использование устойчивых томов. Устойчивый том – место на диске хост-системы, которое монтируется в виртуальную файловую систему образа, и при рестарте сохраняет свое содержимое.

Сеть – это особый сетевой агрегат, позволяющий контейнерам публиковать себя, видеть друг друга и внешние ресурсы. Как правило, контейнеры, которые взаимодействуют друг с другом, должны находиться в одной сети. Обычно это сеть типа мост, то есть инфраструктура Docker создает виртуальные сети, в которые помещает контейнеры, принадлежащие к этой сети.

Dockerfile.

Образ контейнера является конечным артефактом разработки в среде Docker. Контейнер строится по принципу матрешки – выбирается один более общий контейнер и на его основе создается необходимый. Информация о том, что именно должен содержать образ, содержится в специальном манифесте под названием Dockerfile. Манифест используется при построении контейнера и может содержать в себе некоторые команды, за счет которых информирует Docker о том, как надо строить и запускать контейнер.

Манифест представляет собой скрипт, в котором записаны команды, поддерживаемые Docker. Это могут быть следующие команды:

FROM – команда, которая должна быть первой в манифесте. С помощью этой команды устанавливается базовый образ. Также в одном манифесте могут строиться несколько образов, каждый из них будет начинаться с команды FROM. Обычно такая возможность используется для создания отдельных образов для разработки и промышленной эксплуатации.

MAINTAINER – команда, указывающая на то, какое значение должно быть присвоено полю “автор” конечного образа

RUN – команда, позволяющая запустить некоторое приложение в образе в виде обычной строки запуска, как если бы она запускалась через shell.

CMD – команда, позволяющая запустить некоторое приложение в образе в виде массива, первым элементом которого идет приложение, а остальными – параметры. В этой форме запуска не осуществляется обычная обработка shell, то есть, например, $HOME не преобразовывается в домашний каталог.

LABEL – добавляет метку на образ

EXPOSE – инструктирует Docker, что образ ожидает входящих соединений извне по определенному порту

ENV – устанавливает переменную среды

ADD – копирует файлы и директории из источника в файловую систему образа. В отличие от команды COPY может также использовать в качестве источника URL или архив TAR

COPY – делает то же, что и ADD, но без поддержки интернет и архивов

ENTRYPOINT – указывает команду для запуска контейнера. Каждая новая такая команда перетирает предыдущую, поэтому эффект имеет только последняя

VOLUME – создает устойчивое дисковое хранилище для контейнера

USER – устанавливает имя пользователя, от лица которого запускаются команды

WORKDIR – устанавливает рабочий каталог (в файловой системе контейнера)

ARG – устанавливает переменные, которые могут быть использованы как параметры при построении контейнера

ONBUILD – позволяет указать дополнительный манифест, который используется в том случае, если данный образ используется как базовый другим контейнером

STOPSIGNAL – указывает, какие сигналы операционной системы могут приводить к остановке контейнера

HEALTHCHECK – устанавливает правила, по которым Docker решает, корректно ли функционирует контейнер (и, заодно, политики восстановления при нарушении этих правил)

SHELL – устанавливает shell (zsh, ksh,csh, bash и т.д.) который будет использоваться для выполнения последующих команд (вплоть до конца файла или до следующей команды SHELL)

Например, если мы хотим развернуть приложение spring, конечным артефактом которого является файл app-1.0.jar, то соответствующий Dockerfile может выглядеть так:

```
# Dockerfile

FROM openjdk:17

#базовый образ – openjdk 17

RUN mkdir -p /usr/src/myapp

#выполнить команду по созданию каталога на виртуальной файловой системе образа

COPY target/app-1.0.jar /usr/src/myapp

#скопировать артефакт в созданный каталог

WORKDIR /usr/src/myapp
```

```
#установить текущий каталог

ENTRYPOINT ["java", "-jar","app-1.0.jar"]

#установить команду для запуска приложения
```

Разработав контейнер для Docker, программист, скорее всего, хочет автоматически строить его из репозитория. Хотя это возможно с помощью команд непосредственно Docker, в наших примерах, так как мы используем для сборки maven, было бы разумно создавать контейнеры именно им. Фреймворк spring-boot, использованный в примерах, имеет специальную цель для сборки контейнера spring-boot:build-image. С помощью этой цели можно создать контейнер автоматически, при этом в него будут подключены все необходимые зависимости в виде библиотек.

Альтернативно можно использовать плагин docker-maven-plugin от io.fabric8. Этот плагин позволяет описать Dockerfile в виде параметров плагина, после чего можно встраивать процесс построения образа контейнера в жизненный цикл maven.

Docker desktop.

Docker является важным шагом вперед в области виртуализации, однако, и он не свободен от некоторых проблем. Прежде всего, достаточно неудобно управлять контейнерами по одному, так как приложение, обычно, состоит из многих контейнеров. Промышленные системы управления контейнерами, такие как Kubernetes, имеют развитую инфраструктуру и средства управления параметризацией и развертыванием, которые могут использоваться для создания сложных контейнерных агрегатов. В целях разработки, однако, обычно используют более простые инструменты, такие как Docker Desktop.

Docker Desktop использует в своей работе так называемый чарт (chart), который по умолчанию содержится в файле docker-compose.yml. Чарт содержит в себе описание всех используемых

контейнеров, их параметров и взаимосвязей. Например, следующий чарт содержит инструкции о том, как поднять в контейнере базу данных postgres

```
version: "3.9"

services:

  postgres:

    image: postgres:13.3

    environment:

      POSTGRES_DB: "demo"

      POSTGRES_PASSWORD: "pass"

    volumes:

      - .:/docker-entrypoint-initdb.d

    ports:

      - "6432:5432"

  myapp:

    build:

      context: demo

    ports:

      - "9443:8443"

    links:

     - postgres

    depends_on:
```

- postgres

Собственно, информация о запускаемых контейнерах начинается с раздела services. Далее идет имя, которое мы даем сервису (postgres), и базовый имидж. Через имя сервиса другим контейнерам, находящимся в той же сети, будут доступны соединения с этой базой (это имя виртуального хоста). Переменные среды указывают, как будет называться база данных и пароль для пользователя postgres. Раздел volumes нужен для того, чтобы смонтировать скрипт по первичному заполнению базы данных, лежащий в текущем каталоге (все скрипты, лежащие в docker-entrypoint-initdb.d, будут выполнены при первом старте контейнера). Наконец, раздел ports указывает, что порт 5432 виртуального хоста нужно транслировать в порт 6432 хост-машины, исполняющей Docker Desktop.

Следующий сервис – это приложение myapp, которое запускается из контекста demo, в данном случае, каталога с файлом Dockerfile, в котором лежит Java приложение. Разделы links и depends_on указывают Docker Desktop на наличие сетевой связности с базой данных и необходимость стартовать приложение только после старта базы соответственно. Приложение будет доступно извне по порту 9443, куда будет транслирован порт 8443 виртуального хоста.

Запустить такое приложение можно командой docker-compose --build up

Задание на лабораторную работу.

1. Подготовьте Ваше приложение к разворачиванию в облачном сервисе или компоненте Docker.
2. Реализуйте файл docker-compose.xml, который будет содержать все необходимые для работы Вашего приложения ресурсы
3. Внимание. В виду того, что далеко не на каждом компьютере можно запустить систему виртуализации, данная лабораторная работа сдается в электронном виде, без демонстрации преподавателю.

Список литературы.

Интернет-источники.

1. https://spring.io/ [англ.]
2. https://www.baeldung.com [англ.]
3. https://kafka.apache.org/documentation/ [англ.]
4. https://docs.docker.com/ [англ.]
5. https://kubernetes.io/docs/home/ [англ.]